

Глава 8

ОБРАБОТКА СОБЫТИЙ

В этой главе...

- ▼ Общие сведения об обработке событий
- ▼ Иерархия событий библиотеки AWT
- ▼ Семантические и низкоуровневые события в библиотеке AWT
- ▼ Типы низкоуровневых событий
- ▼ Действия
- ▼ Многоадресная передача событий
- ▼ Реализация источников событий



Обработка событий имеет огромное значение для создания программ с графическим пользовательским интерфейсом. Чтобы профессионально реализовать интерфейс программы, необходимо очень хорошо разбираться в этом механизме. В этой главе описана модель событий Java AWT. Вы узнаете, как перехватывать события мыши и клавиатуры, а также использовать простейшие элементы графического пользовательского интерфейса, например кнопки. В частности, здесь обсуждается обработка событий, генерируемых этими компонентами. В следующей главе будет показано, как объединить в одно целое компоненты библиотеки Swing, учитывая весь спектр событий, которые ими генерируются.



В Java 1.1 была предложена новая модель обработки событий, позволяющая создавать эффективно работающие пользовательские интерфейсы. Устаревшая модель события, использованная в Java 1.0, рассматриваться здесь не будет.

Общие сведения об обработке событий

Каждая операционная система, поддерживающая графический пользовательский интерфейс, непрерывно отслеживает такие события, как нажатие клавиш или щелчок мыши, а затем сообщает о них выполняемой программе. Программа решает, как реагировать на эти события (если это предусмотрено ее кодом). В языках, подобных языку Visual Basic, соответствие между событиями и кодом очевидно: среда программирования создает код, реагирующий на определенное событие, и помещает его в так называемую *процедуру обработки события* (event procedure). Например, после щелчка

на кнопке `HelpButton` будет вызвана процедура `HelpButton_Click`. Каждому компоненту пользовательского интерфейса в языке Visual Basic соответствует фиксированное множество событий, состав которого изменить невозможно.

Если для создания программы, реагирующей на события, используется язык, подобный языку C, программист должен создать код, непрерывно проверяющий очередь событий, которая передается ему операционной системой. (Обычно для этого код обработки событий помещается в гигантский цикл с большим количеством операторов `switch`.) Создавать такие программы чрезвычайно трудно. Однако подобный способ имеет преимущество, заключающееся в том, что множество событий ничем не ограничено, в отличие от языка Visual Basic, в котором очередь событий скрыта от программиста.

Подход, принятый в языке Java, представляет собой нечто среднее между подходами, применяемыми в языках Visual Basic и C. Он довольно сложен. При обработке событий, предусмотренных в библиотеке AWT, программист полностью контролирует передачу событий от *источников* (event sources) (например, кнопок или полос прокруток) к *обработчику* (event listener). Любой объект можно определить как обработчик некоего события — на практике все объекты так или иначе реагируют на события. Эта *модель делегирования событий* (event delegation model) позволяет достичь большей гибкости по сравнению с языком Visual Basic, в котором обработчики предопределены. Однако в процессе создания программы нужно написать намного больше кода, разобраться в котором намного сложнее (по крайней мере до тех пор, пока вы не привыкнете к этой технологии).

Источники событий содержат методы, позволяющие связывать их с обработчиками. Когда происходит соответствующее событие, источник извещает всех зарегистрированных обработчиков.

Поскольку язык Java является объектно-ориентированным, информация о событии инкапсулируется в *объекте* события (event object). В языке Java все события описываются подклассами `java.util.EventObject`. В качестве примеров можно привести `ActionEvent` и `WindowEvent`.

Различные источники могут порождать разные виды событий. Например, кнопка может создавать объекты класса `ActionEvent`, а окно — объекты класса `WindowEvent`.

Кратко механизм обработки событий AWT можно описать следующим образом.

- Обработчик представляет собой экземпляр класса, реализующего специальный интерфейс.
- Источник события — это объект, который может регистрировать обработчики и посылать им объекты событий.
- При наступлении события источник посылает объекты события всем зарегистрированным обработчикам.
- Обработчики используют информацию, инкапсулированную в объекте события, для того, чтобы решить, как реагировать на это событие.

Для регистрации обработчика применяется выражение в следующем формате:

```
объект_источника_события.add_событие_Listener(обработчик_события)
```

Рассмотрим следующий фрагмент программы:

```
ActionListener listener = ...;
Jbutton button = new JButton("OK");
button.addActionListener(listener);
```

Теперь обработчик оповещается о событии, связанном с кнопкой. Это может быть, например, простой щелчок.

Код, подобный приведенному выше, требует, чтобы класс обработчика реализовывал соответствующий интерфейс (в данном случае – `ActionListener`). Как и для остальных интерфейсов, реализация предполагает наличие методов, имеющих соответствующую сигнатуру. Для того чтобы реализовать интерфейс `ActionListener`, класс обработчика должен иметь метод с именем `actionPerformed()`, получающий в качестве параметра объект класса `ActionEvent`.

```
class MyListener implements ActionListener
{
    ...
    public void actionPerformed(ActionEvent event)
    {
        // Реакция на активизацию кнопки.
        ...
    }
}
```

Когда пользователь щелкает на кнопке, объект класса `JButton` создает объект класса `ActionEvent` и вызывает метод `listener.actionPerformed(event)`, передавая ему объект события. Обработчиками события (например, щелчка мышью на кнопке) можно назначать и несколько объектов одновременно. В этом случае после щелчка на кнопке объект класса `JButton` вызовет методы `actionPerformed()` всех зарегистрированных обработчиков.

На рис. 8.1 показано взаимодействие источника, обработчика и объекта события.

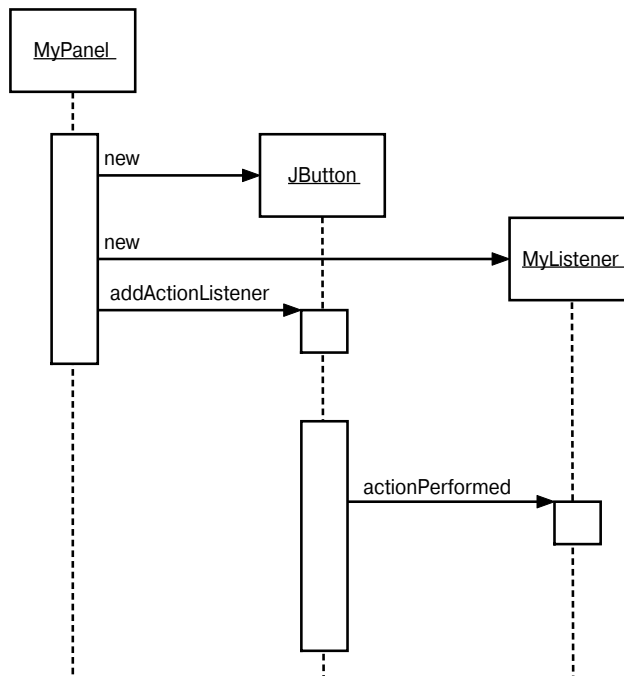


Рис. 8.1. Регистрация и обработка события



В этой главе особое внимание уделено обработке событий, связанных с *пользовательским интерфейсом*, таких как щелчок на кнопке или перемещение курсора мыши. Однако архитектура обработки событий не ограничивается поддержкой пользовательского интерфейса. Как вы увидите в следующей главе, оповещать обработчики могут и объекты, не имеющие никакого отношения к графическому интерфейсу. Обычно в таких случаях события генерируются при изменении свойств объекта.

Пример: обработка щелчка на кнопке

Для того чтобы лучше освоить модель делегирования событий, рассмотрим подробно, как обрабатывается щелчок на кнопке. Для этого нам понадобятся три кнопки, расположенные на панели, и три объекта-обработчика, обеспечивающие реакцию на события, связанные с кнопками.

В рамках этого сценария каждый раз, когда пользователь щелкает мышью на какой-нибудь кнопке, расположенной на панели, соответствующий обработчик получает объект класса `ActionEvent`. В нашей программе в ответ на щелчок будет изменяться цвет панели.

Прежде чем перейти к рассмотрению программы, реагирующей на щелчок кнопки, нужно объяснить, как создаются кнопки и как их размещают на панели. (Более подробно элементы графического пользовательского интерфейса рассматриваются в главе 9.)

Для того чтобы создать кнопку, в ее конструкторе нужно задать метки, пиктограмму или оба параметра одновременно. Рассмотрим две строки кода.

```
JButton yellowButton = new JButton("Yellow");
JButton blueButton = new JButton(new ImageIcon("blue-ball.gif"));
```

Кнопки помещаются на панель с помощью вызова метода с достаточно выразительным именем `add()`. Этот метод получает в качестве параметра указанный компонент и добавляет его в контейнер. Ниже приведен пример, демонстрирующий описанные действия.

```
class ButtonPanel extends JPanel
{
    public ButtonPanel()
    {
        JButton yellowButton = new JButton("Yellow");
        JButton blueButton = new JButton("Blue");
        JButton redButton = new JButton("Red");

        add(yellowButton);
        add(blueButton);
        add(redButton);
    }
}
```

Результат работы этого фрагмента показан на рис. 8.2.

Теперь, поместив кнопки на панель, надо добавить код, позволяющий отслеживать события, происходящие с этими кнопками. Для этого нужен класс, реализующий интерфейс `ActionListener`, в котором, как мы уже говорили, объявлен один метод — `actionPerformed()`. Сигнатура этого метода выглядит следующим образом:

```
public void actionPerformed(ActionEvent event)
```



Рис. 8.2. Панель с кнопками



Интерфейс `ActionListener`, использованный в нашем примере, не ограничивается отслеживанием щелчков на кнопках. Он применяется во многих ситуациях.

- После двойного щелчка на пункте списка.
- После выбора пункта меню.
- После нажатия клавиши `<Enter>`, когда курсор находится в поле редактирования.
- По истечении заданного отрезка времени, отслеживаемого компонентом `Timer`.

Во всех случаях интерфейс `ActionListener` используется совершенно одинаково: метод `actionPerformed()` — единственный метод в интерфейсе `ActionListener` — получает в качестве параметра объект типа `ActionEvent`. Этот объект несет в себе информацию о событии.

Предположим, что мы хотим после щелчка на кнопке изменить цвет панели. Новый цвет нужно указать в классе обработчика.

```
class ColorAction implements Listener
{
    public ColorAction(Color c)
    {
        backgroundColor = c;
    }
    public void actionPerformed(ActionEvent event)
    {
        // Устанавливаем цвет панели.
        ...
    }
    private Color backgroundColor;
}
```

Затем мы создаем по одному объекту для каждого цвета и задаем их в качестве обработчиков событий, связанных с кнопкой.

```
ColorAction yellowAction = new ColorAction(Color.YELLOW);
ColorAction blueAction = new ColorAction(Color.BLUE);
ColorAction redAction = new ColorAction(Color.RED);

yellowButton.addActionListener(yellowAction);
blueButton.addActionListener(blueAction);
redButton.addActionListener(redAction);
```

Например, если пользователь щелкает на кнопке с надписью `Yellow`, вызывается метод `actionPerformed()` объекта `yellowAction`. Поле экземпляра `backgroundColor` хранит значение `Color.YELLOW`, поэтому выполняющийся метод может установить требуемый цвет панели.

Остался один небольшой вопрос. Объект `ColorAction` не имеет доступа к переменной `panel`. Эту проблему можно решить разными способами. Переменную `panel` можно указать в конструкторе класса. Но удобнее сделать класс `ColorAction` внутренним по отношению к классу `ButtonPanel`. В этом случае его методы получают доступ к внешним переменным автоматически. (Более подробно о внутренних классах см. в главе 6.)

Мы будем применять второй способ. Ниже показан пример включения класса `ColorAction` в класс `ButtonPanel`.

```
class ButtonPanel extends JPanel
{
    ...
    private class ColorAction implements ActionListener
    {
        ...
        public void actionPerformed(ActionEvent event)
        {
            setBackground(backgroundColor);
            // Т.е. outer.setBackground(...).
        }
        private Color backgroundColor;
    }
}
```

Рассмотрим метод `actionPerformed()` более подробно. Класс `ColorAction` не имеет методов `setBackground()` и `repaint()`. Однако во внешнем классе `ButtonPanel` эти методы есть. Они вызываются в объекте `ButtonPanel`, создающем объекты внутреннего класса. (Напомним, что слово *outer* не является ключевым словом языка Java. Мы применяем его в качестве интуитивно понятного символа для обозначения ссылки на внешний класс в объекте внутреннего класса.)

Такая ситуация встречается довольно часто. Обработчики должны выполнять какие-то действия, влияющие на другие объекты. Часто бывает очень удобным заранее поместить класс обработчика в состав другого класса, который данный обработчик должен модифицировать.

Листинг 8.1 содержит полный код программы, в которой реализуется обработка событий. Как только пользователь щелкает на какой-нибудь кнопке, соответствующий обработчик изменяет цвет панели.

Листинг 8.1. Содержимое файла `ButtonTest.java`

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ButtonTest
{
    public static void main(String[] args)
    {
        ButtonFrame frame = new ButtonFrame();
    }
}
```

```

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм, содержащий панель с кнопками.
 */
class ButtonFrame extends JFrame
{
    public ButtonFrame()
    {
        setTitle("ButtonTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        ButtonPanel panel = new ButtonPanel();
        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель с тремя кнопками.
 */
class ButtonPanel extends JPanel
{
    public ButtonPanel()
    {
        // Создание кнопок.

        JButton yellowButton = new JButton("Yellow");
        JButton blueButton = new JButton("Blue");
        JButton redButton = new JButton("Red");

        // Включение кнопок в состав панели.

        add(yellowButton);
        add(blueButton);
        add(redButton);

        // Создание обработчиков.

        ColorAction yellowAction = new ColorAction(Color.YELLOW);
        ColorAction blueAction = new ColorAction(Color.BLUE);
        ColorAction redAction = new ColorAction(Color.RED);

        // Связывание обработчиков с кнопками.

        yellowButton.addActionListener(yellowAction);
        blueButton.addActionListener(blueAction);
        redButton.addActionListener(redAction);
    }
}

```

```

/**
 * Обработчик, изменяющий цвет панели.
 */
private class ColorAction implements ActionListener
{
    public ColorAction(Color c)
    {
        backgroundColor = c;
    }

    public void actionPerformed(ActionEvent event)
    {
        setBackground(backgroundColor);
    }

    private Color backgroundColor;
}
}

```



javax.swing.JButton 1.2

- JButton(String label)

Создает кнопку. Строка, передаваемая в качестве параметра, может представлять собой текст или, начиная с версии JDK 1.3, HTML-выражение, например `<HTML><B>OK</B></HTML>`

Параметр label Текст на кнопке

- JButton(Icon icon)

Создает кнопку.

Параметр icon Пиктограмма на кнопке

- JButton(String label, Icon icon)

Создает кнопку.

Параметры label Текст на кнопке
 icon Пиктограмма на кнопке



java.awt.Container 1.0

- Component add(Component c)

Добавляет компонент c в контейнер.



javax.swing.ImageIcon 1.2

- ImageIcon(String filename)

Создает пиктограмму, изображение которой хранится в файле. Это изображение автоматически загружается в объект класса `MediaTracker` (см. главу 7).

Использование внутренних классов в качестве обработчиков событий

Отдельные программисты считают, будто внутренние классы, применяемые в качестве обработчиков, замедляют выполнение программы. Попробуем разобраться в этом. Для каждого компонента пользовательского интерфейса совсем не нужен новый класс. В нашем примере все три кнопки делят между собой один класс-обработчик. Разумеется, каждой из них соответствует отдельный объект этого класса. Однако эти объекты невелики. Такой объект содержит обозначение цвета и ссылку на панель.

Полагая, что время пользоваться внутренними классами уже наступило, мы настоятельно рекомендуем применять именно их в качестве обработчиков событий. В принципе для этого подойдут даже безымянные внутренние классы.

Приведем яркий пример, показывающий, как безымянные внутренние классы позволяют упростить исходный код программы. Вернемся к листингу 8.1. Обратите внимание на то, что обработка событий для всех кнопок совершенно одинакова и состоит из следующих операций.

1. Создание кнопки с соответствующей меткой.
2. Размещение кнопки на панели.
3. Создание обработчика, изменяющего цвет панели.
4. Связывание обработчика с объектом панели.

Чтобы упростить эту задачу, создадим вспомогательный метод.

```
void makeButton(String name, Color backgroundColor)
{
    JButton button = new JButton(name);
    add(button);
    ColorAction action = new ColorAction(backgroundColor);
    button.addActionListener(action);
}
```

Тогда конструктор класса `ButtonPanel` упрощается.

```
public ButtonPanel()
{
    makeButton("yellow", Color.YELLOW);
    makeButton("blue", Color.BLUE);
    makeButton("red", Color.RED);
}
```

Однако возможности упрощения программы этим не исчерпываются. Отметим, что класс `ColorAction` нужен лишь один раз: в методе `makeButton()`. Следовательно, его можно преобразовать в безымянный внутренний класс.

```
void makeButton(String name, final Color backgroundColor)
{
    JButton button = new JButton(name);
    add(button);
    button.addActionListener(new
        ActionListener()
        {
            public void actionPerformed(ActionEvent event)
            {
```

```

        setBackground(backgroundcolor);
    }
});
}

```

Код обработчика упростился. Метод `actionPerformed()` теперь просто ссылается на параметр `backgroundcolor`. (Как и все локальные переменные, доступные во внутреннем классе, этот параметр должен быть терминальным.)

Явный конструктор теперь не нужен. Как мы уже видели в главе 6, механизм внутреннего класса автоматически генерирует конструктор, размещающий в памяти все локальные терминальные переменные, используемые в одном из методов этого класса.



На первый взгляд может показаться, что внутренние классы существенно усложняют код программы. Однако если вы научитесь выделять главное, абстрагируясь на время от остального кода, то легко освоите синтаксис внутренних классов. Рассмотрим пример.

```

button.addActionListener(new
    ActionListener()
    {
        public void actionPerformed(ActionEvent event)
        {
            setBackground(backgroundcolor);
        }
    });

```

Поскольку обработчик события состоит всего лишь из нескольких операторов, мы полагаем, что в этом коде вполне можно разобраться, особенно, если программист не боится применять механизм внутренних классов.



В пакете `JDK 1.4` реализован механизм, позволяющий создавать обработчики событий без программирования внутренних классов. Допустим, например, что приложение имеет кнопку `Load`, обработчик событий которой содержит всего один вызов метода:

```
frame.loadData();
```

Разумеется, можно было бы воспользоваться безымянным внутренним классом.

```

loadButton.addActionListener(new
    ActionListener()
    {
        public void actionPerformed(ActionEvent event)
        {
            frame.loadData();
        }
    });

```

Однако класс `EventHandler` может создать такой обработчик автоматически, вызвав метод `create()`.

```
EventHandler.create(ActionListener.class, frame, "loadData")
```

Разумеется, обработчик по-прежнему необходимо связать с источником событий.

```
loadButton.addActionListener((ActionListener)
    EventHandler.create(ActionListener.class, frame, "loadData"));
```

Приведение типа необходимо, поскольку метод `create()` возвращает объект `Object`. Возможно, в последующих версиях JDK будут использованы универсальные типы, что упростит работу с данным методом.

Если обработчик событий вызывает метод с одним параметром, производным от обработчика событий, можно воспользоваться другой формой метода `create()`. Например, вызов

```
EventHandler.create(ActionListener.class, frame, "loadData",
    "source.text")
```

эквивалентен конструкции

```
new ActionListener()
{
    public void actionPerformed(ActionEvent event)
    {
        frame.loadData(
            ((JTextField)event.getSource()).getText());
    }
}
```

Обратите внимание на то, что обработчик событий превращает имена свойств `source` и `text` в вызовы методов `getSource()` и `getText()`, используя соглашения, принятые для компонентов JavaBeans. (Более полная информация о свойствах компонентов JavaBeans содержится во втором томе.)

Однако на практике эта ситуация встречается редко, и механизма поддержки параметров, не являющихся объектами классов, производных от классов событий, не существует.

Превращение компонентов в обработчики событий

Программист может совершенно свободно назначать обработчиком события, связанного с кнопкой, *любой* объект класса, реализующего интерфейс `ActionListener`. В нашем примере используются объекты нового класса, созданного специально для выполнения требуемых действий. Многие поступают иначе и добавляют метод `actionPerformed()` в существующий класс. Разумеется, этот класс также должен реализовывать интерфейс `ActionListener`. Например, можно легко превратить в обработчик класс `ButtonPanel`.

```
class ButtonPanel extends JPanel
    implements ActionListener
{
    ...
    public void actionPerformed(ActionEvent event)
    {
        // Устанавливает цвет фона.
```

```

    ...
}
}

```

В этом случае панель сама себя назначает обработчиком событий для всех трех кнопок.

```

yellowButton.addActionListener(this);
blueButton.addActionListener(this);
redButton.addActionListener(this);

```

Отметим, что теперь три кнопки больше не имеют индивидуальных обработчиков. У них есть общий обработчик — панель, на которой они расположены. Следовательно, метод `actionPerformed()` должен как-то распознавать, на *какой* кнопке произведен щелчок.

Источник любого события распознается с помощью метода `getSource()` класса `EventObject` — суперкласса для всех событий. Источник события — это объект, генерирующий событие и уведомляющий об этом обработчик.

```
Object source = event.getSource();
```

После вызова данного метода `actionPerformed()` проверяет, какая из кнопок была источником события.

```

if (source == yellowButton) ...
else if (source == blueButton) ...
else if (source == redButton) ...

```

Разумеется, при этом нужно, чтобы все ссылки на кнопки хранились в полях содержащей их панели.



Некоторые программисты пользуются другим способом распознавания источника. В классе `ActionEvent` есть метод `getActionCommand()`, который возвращает *командную строку*, связанную с данным действием. Для кнопки он по умолчанию возвращает ее метку. Если вы предпочитаете такой способ, то метод `actionPerformed()` должен содержать следующий фрагмент кода (или подобный ему):

```

String command = event.getActionCommand();
if (command.equals("Yellow")) ...;
else if (command.equals("Blue")) ...;
else if (command.equals("Red")) ...;

```

Тем не менее полагаться на метки кнопок небезопасно. Легко сделать ошибку и назвать кнопку "Gray", а затем попытаться проверить источник события с помощью оператора

```
if (command.equals("Grey")) ...
```

Кроме того, при интернационализации вашей программы придется для каждого отдельного языка изменять название кнопки. Например, в немецком языке кнопки будут носить названия "Gelb", "Blau" и "Rot". В каждом из этих случаев программист будет вынужден изменять как метку кнопки, так и параметр метода `actionPerformed()`.



java.util.EventObject 1.1

- `Object getSource()`
Возвращает ссылку на объект, являющийся источником события.

**java.awt.event.ActionEvent 1.1**

- `String getActionCommand()`

Возвращает командную строку, связанную с событием. Если событие произошло с кнопкой, командной строкой является метка строки (заметьте, что она может быть изменена с помощью метода `setActionCommand()`).

**java.beans.EventHandler 1.4**

- `static Object create(Class listenerInterface, Object target, String action)`
- `static Object create(Class listenerInterface, Object target, String action, String eventProperty)`
- `static Object create(Class listenerInterface, Object target, String action, String eventProperty, String listenerMethod)`

Эти методы создают proxy-объект, реализующий заданный интерфейс. В результате либо указанный метод, либо все методы интерфейса выполняют заданное действие с целевым объектом.

Параметр `action` может быть именем метода или свойством объекта. Если это свойство — выполняется метод `set...()`. Например, если параметр `action` имеет значение `text`, оно превращается в вызов метода `setText()`.

Свойство события состоит из одного или нескольких имен свойств, разделенных точками. Первое свойство задает считывание параметра метода из обработчика, второе свойство — считывание результирующего объекта и т.д. Например, свойство `"source.text"` превращается в вызовы методов `getSource()` и `getText()`.

Пример: изменение стиля

По умолчанию программы, созданные с помощью пакета `Swing`, используют стиль `Metal`. Изменить стиль можно двумя способами. В подкаталоге `JDK jre/lib` можно предусмотреть файл `swing.properties` и задать в нем с помощью свойства `swing.defaultlaf` имя класса, определяющего нужный стиль, например:

```
swing.defaultlaf=com.sun.java.swing.plaf.motif.MotifLookAndFeel
```

Отметим, что стиль `Metal` описан в пакете `javax.swing`. Другие стили описаны в пакете `com.sun.java`. Они не являются обязательным атрибутом каждой реализации языка `Java`. В данный момент по соображениям, связанным с авторским правом, пакеты стилей для операционных систем `Windows` и `Mac` поставляются только с версиями `JDK`, ориентированными на эти системы.



Поскольку строки, начинающиеся символом `#`, в файлах, описывающих свойства стилей, игнорируются, в файле `swing.properties` можно предусмотреть несколько стилей, “закомментировав” ненужные.

```
#swing.defaultlaf=javax.swing.plaf.metal.MetalLookAndFeel
swing.defaultlaf=com.sun.java.swing.plaf.motif.MotifLookAndFeel
#swing.defaultlaf=com.sun.java.swing.plaf.windows.WindowsLookAndFeel
```

Чтобы изменить стиль интерфейса программы, ее нужно запустить заново. Программа прочитывает файл `swing.properties` лишь один раз, при запуске.

Для того чтобы изменять стиль динамически, вызовите метод `UIManager.setLookAndFeel()` и задайте ему имя требуемого стиля. Затем вызовите статический метод `SwingUtilities.updateComponentTreeUI()` и обновите с его помощью весь набор компонентов. Этому методу достаточно передать один компонент, остальные он найдет автоматически. Если нужный стиль не будет обнаружен или произойдет ошибка при его загрузке, метод `UIManager.setLookAndFeel` может сгенерировать различные исключения. Как обычно, вы можете не уделять внимания обработке исключений, поскольку эта тема будет полностью освещена в главе 11.

Рассмотрим пример, в котором задается стиль Motif графического интерфейса программы.

```
String plaf = "com.sun.java.swing.plaf.motif.MotifLookAndFeel";
try
{
    UIManager.setLookAndFeel(plaf);
    SwingUtilities.updateComponentTreeUI(panel);
}
catch(Exception e) { e.printStackTrace(); }
```

Для того чтобы пронумеровать установленные реализации стилей, надо использовать следующее выражение:

```
UIManager.LookAndFeelInfo[] infos =
    UIManager.getInstalledLookAndFeels();
```

После этого вы можете с помощью приведенного ниже кода получить имя каждого стиля и реализующего его класса.

```
String name = infos[i].getName();
String className = infos[i].getClassName();
```

В листинге 8.2 приведена законченная программа, демонстрирующая переключение стилей (рис. 8.3). Эта программа очень похожа на ту, которая приведена в листинге 8.1. Следуя советам, изложенным выше, для того, чтобы задать действия с кнопками, т.е. переключение стилей, мы применили вспомогательный метод `makeButton()` и безымянный внутренний класс.



Рис. 8.3. Переключение стилей

У этой программы есть одно преимущество. Метод `actionPerformed()` во внутреннем классе-обработчике должен передать методу `updateComponentTreeUI()` ссылку `this` на объект внешнего класса `PlafPanel`. Напомним, что ссылка `this` на объект внешнего класса должна сопровождаться префиксом — именем этого класса.

`SwingUtilities.updateComponentTreeUI(PlafPanel.this)`

Листинг 8.2. Содержимое файла `PlafTest.java`

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class PlafTest
{
    public static void main(String[] args)
    {
        PlafFrame frame = new PlafFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм с панелью, содержащей кнопки. С помощью
 * кнопок изменяется стиль окна.
 */
class PlafFrame extends JFrame
{
    public PlafFrame()
    {
        setTitle("PlafTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        PlafPanel panel = new PlafPanel();
        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель с кнопками, предназначенными для изменения стиля
 */
class PlafPanel extends JPanel
{
    public PlafPanel()
    {
        UIManager.LookAndFeelInfo[] infos =
            UIManager.getInstalledLookAndFeels();
        for (UIManager.LookAndFeelInfo info : infos)
            makeButton(info.getName(), info.getClassName());
    }
}
```

```

/**
 * Создает кнопки, изменяющие стиль отображения.
 * @param name Имя кнопки
 * @param plafName Имя класса, который описывает стиль
 */
void makeButton(String name, final String plafName)
{
    // Включение кнопки в состав панели.

    JButton button = new JButton(name);
    add(button);

    // Связывание обработчика событий с кнопкой.

    button.addActionListener(new
        ActionListener()
        {
            public void actionPerformed(ActionEvent event)
            {
                // Действие: переключение на другой стиль.
                try
                {
                    UIManager.setLookAndFeel(plafName);
                    SwingUtilities.updateComponentTreeUI
                        (PlafPanel.this);
                }
                catch(Exception e) { e.printStackTrace(); }
            }
        });
}
}

```



javax.swing.UIManager 1.2

- static UIManager.LookAndFeelInfo[] getInstalledLookAndFeels()
Получение массива объектов, описывающих реализации установленных стилей.
- static setLookAndFeel(String className)
Установка текущего стиля.
Параметр className Имя, класса, реализующего стиль



javax.swing.UIManager.LookAndFeelInfo 1.2

- String getName()
Возвращает имя стиля.
- String getClassName()
Возвращает имя класса, реализующего стиль.

Пример: обработка событий, связанных с окнами

Не все события обрабатываются так же просто, как события, связанные с кнопками. Рассмотрим более сложный случай, о котором мы уже упоминали в главе 7. До появления `EXIT_ON_CLOSE` в пакете `JDK 1.3` программисты должны были вручную реализовывать выход из программы при закрытии главного фрейма. Если программа представляет собой профессиональное приложение, она обязательно должна завершаться корректным, и пользователь должен иметь гарантию, что он не потеряет результаты выполненной работы. Например, если пользователь закрыл фрейм, можно вывести на экран диалоговое окно и предупредить его о необходимости сохранить результаты работы и только после его согласия завершить выполнение приложения.

Если пользователь пытается закрыть фрейм, объект `JFrame` становится источником события `WindowEvent`. Для того чтобы обработать это событие, нужен соответствующий объект, который следует зарегистрировать известным вам образом.

```
WindowListener listener = ...;
frame.addWindowListener(listener);
```

В данном случае обработчик должен быть экземпляром класса, реализующего интерфейс `WindowListener`. В интерфейсе `WindowListener` есть семь методов. Фрейм вызывает их в ответ на семь разных событий, которые могут произойти с окном. Их имена отражают их назначение. Исключением, может быть, является слово `iconified`, что в системе `Windows` означает “свернутое” окно. Вот как выглядит полный интерфейс `WindowListener`.

```
public interface WindowListener
{
    void windowOpened(WindowEvent e);
    void windowClosing(WindowEvent e);
    void windowClosed(WindowEvent e);
    void windowIconified(WindowEvent e);
    void windowDeiconified(WindowEvent e);
    void windowActivated(WindowEvent e);
    void windowDeactivated(WindowEvent e);
}
```



Если нужно проверить, было ли окно увеличено до максимальных размеров, следует использовать обработчик типа `WindowStateListener`. Подробную информацию о нем можно найти в документации на API.

Как обычно, любой класс, реализующий какой-либо интерфейс, должен реализовывать все его методы. В данном случае это означает создание тела *семи* методов. Напомним, что нас интересует только один из них — метод `windowClosing()`.

Разумеется, можно определить класс, реализующий интерфейс `WindowListener`, добавить в метод `windowClosing()` вызов `System.exit(0)` и написать пустые функции для остальных шести методов.

```
class Terminator implements WindowListener
{
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
}
```

```

    public void windowOpened(WindowEvent e) {}
    public void windowClosing(WindowEvent e) {}
    public void windowClosed(WindowEvent e) {}
    public void windowIconified(WindowEvent e) {}
    public void windowDeiconified(WindowEvent e) {}
    public void windowActivated(WindowEvent e) {}
    public void windowDeactivated(WindowEvent e) {}
}

```

Классы-адаптеры

Создавать код для шести методов, которые ничего не делают, — неблагодарное занятие. Чтобы упростить нашу задачу, для каждого из интерфейсов обработчиков, в котором содержится несколько методов, создается *класс-адаптер*, реализующий все эти методы, причем тело их пусто. Например, класс `WindowAdapter` содержит семь методов, не выполняющих никаких операций. Следовательно, класс-адаптер автоматически удовлетворит требованиям, предъявляемым к реализации соответствующего интерфейса. Программист может расширить класс-адаптер и уточнить нужные реакции на некоторые и даже на все типы событий, происходящих в интерфейсе. (Заметьте, что интерфейс `ActionListener` содержит только один метод, поэтому для него класс-адаптер не нужен.)

Рассмотрим применение адаптера для окна. Расширим класс `WindowAdapter`, унаследовав шесть пустых методов и заменив метод `windowClosing()`.

```

class Terminator extends WindowAdapter
{
    public void windowClosing(WindowEvent e)
    {
        System.exit(0);
    }
}

```

Теперь объект класса `Terminator` можно зарегистрировать как обработчик событий.

```

WindowListener listener = new Terminator();
frame.addWindowListener(listener);

```

С этого момента, как только фрейм сгенерирует событие, он передаст его обработчик, вызвав один из его семи методов (рис. 8.4). Шесть из этих методов ничего не делают, а метод `windowClosing()` выполняет вызов `System.exit(0)`, завершая работу приложения.



Если, расширяя класс адаптера, вы неправильно укажете имя метода, компилятор не сообщит об ошибке. Например, если в подклассе `WindowAdapter` вы создадите метод `windowIsClosing()`, то получите новый класс, содержащий восемь методов, причем метод `windowClosing()` не будет выполнять никаких действий.

Создание класса обработчика, расширяющего класс `WindowAdapter`, — несомненный шаг вперед, но можно достичь еще большего. Объекту обработчика не обязательно давать имя, достаточно написать следующее выражение:

```

frame.addWindowListener(new Terminator());

```

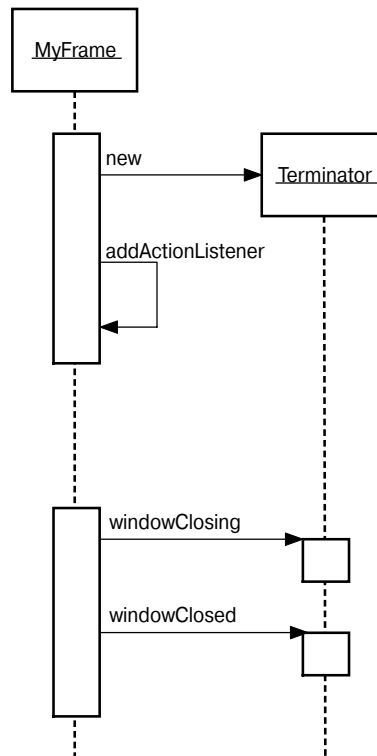


Рис. 8.4. Обработка событий, связанных с окном

Но и это еще не все! Можно использовать в качестве обработчика безымянный внутренний класс фрейма.

```

frame.addWindowListener(new
    WindowAdapter()
    {
        public void windowClosing(WindowEvent e)
        {
            System.exit(0);
        }
    }
);
  
```

В этом фрагменте кода выполняются следующие действия.

- Определяется безымянный класс, расширяющий класс `WindowAdapter`.
- В этом безымянном классе переопределяется метод `windowClosing()`. Как и прежде, данный метод отвечает за выход из программы.
- Новый класс наследует от класса `WindowAdapter` шесть методов, не выполняющих никаких действий.
- Создается объект класса. Этот объект также не имеет имени.
- Безымянный объект передается методу `addWindowListener()`.

**java.awt.event.WindowListener 1.1**

- **void windowOpened(WindowEvent e)**
Этот метод вызывается после открытия окна.
- **void windowClosing(WindowEvent e)**
Этот метод вызывается, когда пользователь отдает оконному диспетчеру команду закрыть окно. Обратите внимание на то, что окно закрывается, только если вызван метод `hide()` или `dispose()`.
- **void windowClosed(WindowEvent e)**
Этот метод вызывается после закрытия окна.
- **void windowIconified(WindowEvent e)**
Этот метод вызывается после свертывания окна.
- **void windowDeiconified(WindowEvent e)**
Этот метод вызывается после развертывания окна.
- **void windowActivated(WindowEvent e)**
Этот метод вызывается после активизации окна. Активным может быть только фрейм или диалоговое окно. Обычно оконный диспетчер специально отмечает активное окно, например подсвечивает его заголовок.
- **void windowDeactivated(WindowEvent e)**
Этот метод вызывается после того, как окно стало неактивным.

**java.awt.event.WindowStateListener 1.4**

- **void windowStateChanged(WindowEvent event) 1.4**
Этот метод вызывается после того, как окно было увеличено до максимальных размеров, свернуто или если были восстановлены его нормальные размеры.

**java.awt.event.WindowEvent 1.1**

- **int getNewState() 1.4**
- **int getOldState() 1.4**

Эти методы возвращают новое или старое состояние окна при изменении его состояния. Возвращаемое целое число может принимать одно из следующих значений:

```
Frame.NORMAL
Frame.ICONIFIED
Frame.MAXIMIZED_HORIZ
Frame.MAXIMIZED_VERT
Frame.MAXIMIZED_BOTH
```

Иерархия событий библиотеки AWT

После обсуждения механизма обработки событий перейдем к более общим вопросам. Как уже указывалось ранее, в языке Java при обработке событий используется объектный подход. Все события являются потомками класса `EventObject` из пакета `java.util`. (Общий суперкласс не назвали именем `Event`, так как это имя носил класс событий в старой модели. Хотя старая модель в настоящее время не рекомендована к использованию, ее классы все еще являются частью библиотеки Java.)

Класс `EventObject` имеет подкласс `AWTEvent`, являющийся родительским по отношению ко всем классам событий из пакета AWT. На рис. 8.5 показана диаграмма наследования событий из пакета AWT.

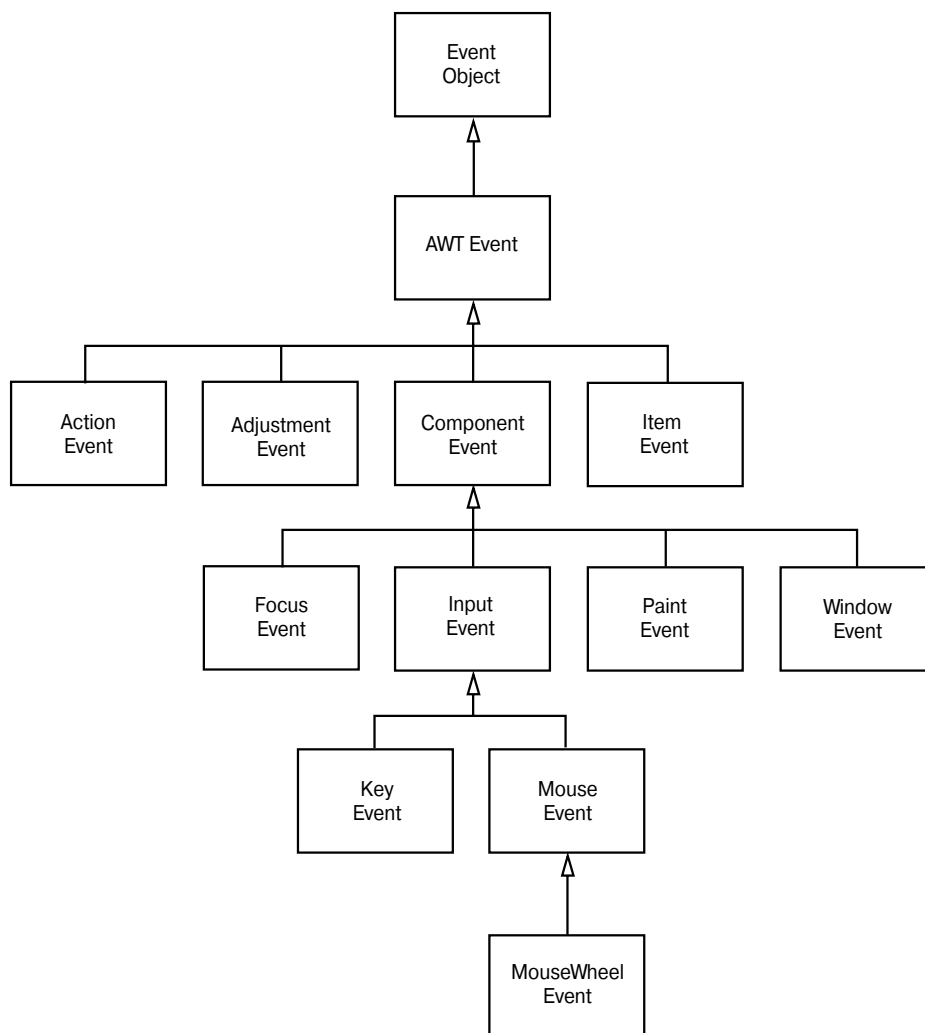


Рис. 8.5. Диаграмма наследования классов событий AWT

Некоторые компоненты пакета Swing генерируют другие объекты событий и непосредственно расширяют класс `EventObject`.

Объекты событий инкапсулируют информацию, которую источник события передает обработчику. По мере необходимости объекты событий можно проанализировать с помощью методов `getSource()` и `getActionSource()`.

Некоторые из классов событий AWT не имеют практической ценности. Например, пакет AWT помещает объекты `PaintEvent` в очередь событий, однако эти объекты обработчику не передаются. Программисты должны сами переопределять метод `paintComponent()`, чтобы управлять перерисовкой. AWT также генерирует ряд событий, интересующих лишь системных программистов; они могут использоваться для поддержки иероглифических языков, автоматизации тестирования роботов и выполнения других самых разнообразных задач. В этой книге мы не будем обсуждать специализированные типы событий, а также обойдем вниманием события, связанные с устаревшими и не используемыми сейчас компонентами AWT.

Ниже перечислены часто применяемые события AWT.

<code>ActionEvent</code>	<code>KeyEvent</code>
<code>AdjustmentEvent</code>	<code>MouseEvent</code>
<code>FocusEvent</code>	<code>MouseEvent</code>
<code>ItemEvent</code>	<code>WindowEvent</code>

В этой и последующих главах будут приведены примеры, иллюстрирующие использование всех указанных типов событий.

Пакет `javax.swing.event` содержит дополнительные события, характерные для компонентов пакета Swing. Некоторые из них будут описаны в следующей главе.

Интерфейсы, используемые для создания обработчиков событий, имеют следующие имена.

<code>ActionListener</code>	<code>MouseMotionListener</code>
<code>AdjustmentListener</code>	<code>MouseListener</code>
<code>FocusListener</code>	<code>WindowListener</code>
<code>ItemListener</code>	<code>WindowFocusListener</code>
<code>KeyListener</code>	<code>WindowStateListener</code>
<code>MouseListener</code>	

С интерфейсами `ActionListener` и `WindowListener` вы уже знакомы. Они использовались в рассмотренных выше примерах.

Несмотря на то что пакет `javax.swing.event` содержит очень много интерфейсов, специфичных для компонентов из пакета Swing, в нем все еще используются основные интерфейсы из библиотеки AWT. В частности, они применяются для обработки событий общего характера.

Семь интерфейсов из пакета AWT, имеющих несколько методов, сопровождаются классами-адаптерами, реализующими пустые методы. (Оставшиеся интерфейсы содержат только по одному методу, поэтому для них классы-адаптеры не нужны.) Ниже перечислены имена классов-адаптеров.

<code>FocusAdapter</code>	<code>MouseMotionAdapter</code>
<code>KeyAdapter</code>	<code>WindowAdapter</code>
<code>MouseAdapter</code>	

В каждой прикладной программе используется много классов и интерфейсов, заслуживающих внимания. Класс, предназначенный для получения информации о событии, должен реализовывать интерфейс обработчика. Объект этого класса связывается с источником события. При возникновении события оно передается объекту, где обрабатывается с помощью методов, определенных в интерфейсе обработчика.



Программисты, работавшие на языках C и C++, могут поинтересоваться: зачем для обработки событий нужны все эти хитросплетения объектов, методов и интерфейсов? Ведь для создания графического пользовательского интерфейса на языке C++ можно использовать обратные вызовы с обобщенными указателями или обработчиками. Однако в языке Java этот механизм не работает. Модель обработки событий в языке Java строго типизирована: компилятор следит, чтобы события передавались только объектам, предназначенным для их обработки.

Семантические и низкоуровневые события в библиотеке AWT

В AWT события разделены на *низкоуровневые* (low-level) и *семантические* (semantic). Семантические события описывают действия пользователя, например щелчок на кнопке; следовательно, событие `ActionEvent` является семантическим. Низкоуровневые события обеспечивают возможность таких действий. Если пользователь щелкнул на кнопке, значит, он нажал кнопку мыши, возможно, переместил курсор по экрану и отпустил кнопку мыши (причем курсор мыши должен находиться в пределах кнопки). Семантические события могут быть также сгенерированы путем нажатия клавиш, например для перемещения по кнопкам с помощью клавиши `<Tab>`. Аналогично, действие с полосой прокрутки относится к семантическим событиям, в то время как перетаскивание объекта с помощью мыши относится к низкоуровневым.

Из классов пакета `java.awt.event`, описывающих семантические события, наиболее часто используются следующие.

- `ActionEvent` (щелчок на кнопке, выбор пункта меню, выбор пункта в списке, нажатие клавиши `<Enter>` при работе с полем редактирования).
- `AdjustmentEvent` (перемещение ползунка на полосе прокрутки).
- `ItemEvent` (выбор положения переключателя опций или пункта в списке).

Из низкоуровневых событий наиболее часто встречаются перечисленные ниже.

- `KeyEvent` (нажатие или отпускание клавиши).
- `MouseEvent` (нажатие, отпускание кнопки мыши, перемещение курсора мыши, перетаскивание курсора, т.е. его перемещение при нажатой кнопке).
- `MouseWheelEvent` (вращение колесика мыши).
- `FocusEvent` (получение или потеря фокуса ввода.).
- `WindowEvent` (изменение состояния окна).

В табл. 8.1 приведены наиболее важные интерфейсы обработчиков, события и источники событий.

Таблица 8.1. Объекты и интерфейсы, участвующие в обработке событий

Интерфейс	Методы	Параметр/ Методы доступа	Источник события
ActionListener	actionPerformed	ActionEvent • getActionCommand • getModifiers	AbstractButton JComboBox JTextField Timer
AdjustmentListener	adjustmentValueChanged	AdjustmentEvent • getAdjustable • getAdjustmentType • getValue	JScrollbar
ItemListener	itemStateChanged	ItemEvent • getItem • getItemSelectable • getStateChange	AbstractButton JComboBox
FocusListener	focusGained focusLost	FocusEvent • isTemporary	Component
KeyListener	keyPressed keyReleased keyTyped	KeyEvent • getKeyChar • getKeyCode • getKeyModifiersText • getKeyText • isActionKey	Component
MouseListener	mousePressed mouseReleased mouseEntered mouseExited mouseClicked	MouseEvent • getClickCount • getX • getY • getPoint • translatePoint	Component
MouseMotionListener	mouseDragged mouseMoved	MouseEvent	Component
MouseWheelListener	mouseWheelMoved	MouseWheelEvent • getWheelRotation • getScrollAmount	Component
WindowListener	windowClosing windowOpened windowIconified windowDeiconified windowClosed windowActivated windowDeactivated	WindowEvent • getWindow	Window
WindowFocusListener	windowGainedFocus windowLostFocus	WindowEvent • getOppositeWindow	Window
WindowStateListener	windowStateChanged	WindowEvent • getOldState • getNewState	Window

Взаимосвязь основных понятий, используемых при обработке событий

Рассмотрим еще раз механизм делегирования событий, чтобы убедиться, что вы правильно понимаете отношения между классами событий, интерфейсами обработчиков и классами-адаптерами.

Источники событий — это компоненты пользовательского интерфейса, окна и меню. Операционная система извещает источник события о поведении интересующих его объектов, например о перемещении мыши или нажатии клавиши. Источник события описывает природу события в *объекте* события. Кроме того, он хранит набор *обработчиков* — объектов, которые следует вызвать при наступлении определенного события (рис. 8.6). Затем источник события вызывает соответствующий метод, объявленный в *интерфейсе обработчика*, для того, чтобы доставить информацию о событии требуемому объекту. С этой целью источник передает объект события методу обработчика. Обработчик анализирует объект события, извлекая из него информацию о выполненном действии. Например, чтобы обнаружить источник события, можно вызвать метод `getSource()`, а для того чтобы определить местонахождение курсора мыши — методы `getX()` и `getY()`.



Рис. 8.6. Взаимосвязь между источниками событий и обработчиками

Обратите внимание на то, что интерфейсы `MouseListener` и `MouseMotionListener` отличаются друг от друга. Это сделано для повышения эффективности — есть много событий, связанных с мышью, которые не требуют реакции, например бесцельное перемещение курсора мыши по экрану. Поэтому часто обработчик должен заботиться лишь о нажатии кнопки мыши и не обращать внимания на ее произвольные перемещения.

Все низкоуровневые события являются потомками класса `ComponentEvent`. Этот класс содержит метод `getComponent()`, сообщающий о компоненте, породившем данное событие. Этот метод можно использовать вместо метода `getSource()`. Он возвращает то же значение, что и метод `getSource()`, но при этом возвращаемое значение приводится к типу `Component`. Например, если при вводе текста произойдет событие, связанное с нажатием клавиши, метод `getComponent()` вернет ссылку на соответствующее поле ввода.



java.awt.event.ComponentEvent 1.0

- `Component getComponent()`

Возвращает ссылку на компонент, являющийся источником события. Точно такая же ссылка возвращается при выполнении выражения `(Component) getSources()`.

Типы низкоуровневых событий

В этом разделе мы детально обсудим события, не связанные с конкретными компонентами пользовательского интерфейса, в частности, события, которые возникают при работе с клавиатурой и мышью. Подробный анализ семантических событий, порождаемых компонентами графического пользовательского интерфейса, проводится в следующей главе.

События, связанные с клавиатурой

Нажимая клавишу, пользователь порождает событие `KEY_PRESSED`, а отпуская ее — `KEY_RELEASED`. Оба этих события принадлежат классу `KeyEvent`. Их можно перехватить с помощью методов `keyPressed()` и `keyReleased()` любого класса, реализующего интерфейс `KeyListener`. В частности, эти методы можно использовать для отслеживания ошибочно нажатых клавиш. Третий метод, `keyTyped`, представляет собой сочетание двух предыдущих: он сообщает, какой символ соответствует нажатой клавише.

Перед тем как переходить к рассмотрению конкретного примера, уточним терминологию. В языке Java существует различие между символами (*symbols*) и виртуальными кодами клавиш (*virtual key codes*). Виртуальные коды клавиш обозначаются с помощью префикса `VK_`, например `VK_A` или `VK_SHIFT`. Они соответствуют клавишам клавиатуры; так, код `VK_A` означает клавишу `<A>`. Виртуальный код соответствует клавише, прописные и строчные буквы не различаются.



Виртуальные коды клавиш соответствуют их сканкодам.

Предположим, что пользователь ввел прописную букву “А”, нажав клавишу `<Shift>` и клавишу `<A>`. При этом происходит *пять* событий.

1. Нажатие клавиши `<Shift>` (вызывается метод `keyPressed()` с параметром `VK_SHIFT`).
2. Нажатие клавиши `<A>` (вызывается метод `keyPressed()` с параметром `VK_A`).
3. Ввод буквы “А” (вызывается метод `keyTyped()` с параметром “А”).
4. Отпускание клавиши `<A>` (вызывается метод `keyReleased()` с параметром `VK_A`).
5. Отпускание клавиши `<Shift>` (вызывается метод `keyReleased()` с параметром `VK_SHIFT`).

Если же пользователь вводит строчную букву “а”, нажимая клавишу `<A>`, произойдут только три события.

1. Нажатие клавиши `<A>` (вызывается метод `keyPressed()` с параметром `VK_A`).
2. Ввод буквы “а” (вызывается метод `keyTyped()` с параметром “а”).
3. Отпускание клавиши `<A>` (вызывается метод `keyReleased()` с параметром `VK_A`).

Итак, процедура `keyTyped()` сообщает, какой символ был введен (“А” или “а”), в то время как методы `keyPressed()` и `keyReleased()` сообщают, какие клавиши были нажаты.

При работе с методами `keyPressed` и `keyReleased` нужно сначала проверить код клавиши.

```
public void keyPressed(KeyEvent event)
{
    int keyCode = event.getKeyCode();
    ...
}
```

Код клавиши равен одной из следующих констант (их имена вполне очевидны). Эти константы определены в классе `KeyEvent`.

```
VK_A . . . VK_Z
VK_0 . . . VK_9
VK_COMMA, VK_PERIOD, VK_SLASH, VK_SEMICOLON, VK_EQUALS
VK_OPEN_BRACKET, VK_BACK_SLASH, VK_CLOSE_BRACKET
VK_BACK_QUOTE, VK_QUOTE
VK_GREATER, VK_LESS, VK_UNDERSCORE, VK_MINUS
VK_AMPERSAND, VK_ASTERISK, VK_AT, VK_BRACELEFT, VK_BRACERIGHT
VK_LEFT_PARENTHESIS, VK_RIGHT_PARENTHESIS
VK_CIRCUMFLEX, VK_COLON, VK_NUMBER_SIGN, VK_QUOTEDBL
VK_EXCLAMATION_MARK, VK_INVERTED_EXCLAMATION_MARK
VK_DEAD_ABOVEDOT, VK_DEAD_ABOVERING, VK_DEAD_ACUTE
VK_DEAD_BREVE
VK_DEAD_CARON, VK_DEAD_CEDILLA, VK_DEAD_CIRCUMFLEX
VK_DEAD_DIAERESIS
VK_DEAD_DOUBLEACUTE, VK_DEAD_GRAVE, VK_DEAD_IOTA, VK_DEAD_MACRON
VK_DEAD_OGONEK, VK_DEAD_SEMIVOICED_SOUND, VK_DEAD_TILDE,
VK_DEAD_VOICED_SOUND
VK_DOLLAR, VK_EURO_SIGN
VK_SPACE, VK_ENTER, VK_BACK_SPACE, VK_TAB, VK_ESCAPE
VK_SHIFT, VK_CONTROL, VK_ALT, VK_ALT_GRAPH, VK_META
VK_NUM_LOCK, VK_SCROLL_LOCK, VK_CAPS_LOCK
VK_PAUSE, VK_PRINTSCREEN
VK_PAGE_UP, VK_PAGE_DOWN, VK_END, VK_HOME, VK_LEFT, VK_UP, VK_RIGHT,
VK_DOWN
VK_F1 . . . VK_F24
VK_NUMPAD0 . . . VK_NUMPAD9
VK_KP_DOWN, VK_KP_LEFT, VK_KP_RIGHT, VK_KP_UP
VK_MULTIPLY, VK_ADD, VK_SEPARATOR [sic], VK_SUBTRACT, VK_DECIMAL,
VK_DIVIDE
VK_DELETE, VK_INSERT
VK_HELP, VK_CANCEL, VK_CLEAR, VK_FINAL
VK_CONVERT, VK_NONCONVERT, VK_ACCEPT, VK_MODECHANGE
VK_AGAIN, VK_ALPHANUMERIC, VK_CODE_INPUT, VK_COMPOSE, VK_PROPS
VK_STOP
VK_ALL_CANDIDATES, VK_PREVIOUS_CANDIDATE
VK_COPY, VK_CUT, VK_PASTE, VK_UNDO
VK_FULL_WIDTH, VK_HALF_WIDTH
VK_HIRAGANA, VK_KATAKANA, VK_ROMAN_CHARACTERS
VK_KANA, VK_KANJI
VK_JAPANESE_HIRAGANA, VK_JAPANESE_KATAKANA, VK_JAPANESE_ROMAN
VK_WINDOWS, VK_CONTEXT_MENU
VK_UNDEFINED
```

Чтобы определить текущее состояние клавиш <Shift>, <Control>, <Alt> и <Meta>, можно, конечно, отслеживать коды `VK_SHIFT`, `VK_CONTROL` и `VK_META`, но это очень сложно. Достаточно использовать методы `isShiftDown()`, `isControlDown()`, `isAltDown()` и `isMetaDown()`.

Например, следующий фрагмент кода проверяет, нажимается ли клавиша со стрелкой вправо при нажатой клавише <Shift>.

```
public void keyPressed(KeyEvent event)
{
    int keyCode == event.getKeyCode();
    if (keyCode == KeyEvent.VK_RIGHT && event.isShiftDown())
    {
        ...
    }
}
```

В методе `keyTyped()` вызывается метод `getKeyChar()`, позволяющий определить, какой символ был введен.



Метод `keyTyped()` можно применять не для всех клавиш. Он предназначен лишь для отслеживания таких клавиш, которые генерируют символ в кодировке Unicode. Чтобы проверять действия с управляющими клавишами, например вызывающими перемещение курсора, надо использовать метод `keyPressed()`.

Программа, представленная в листинге 8.3, иллюстрирует обработку нажатия клавиш. Результат ее работы показан на рис. 8.7. Программа представляет собой простую реализацию игрушки Etch-A-Sketch™ и позволяет создавать простые эскизы.

Перо можно перемещать вверх, вниз, влево и вправо, пользуясь клавишами со стрелками. Если при этом нажать клавишу <Shift>, перо переместится на большее расстояние. Тот, кто имеет опыт работы с редактором Vi, может переназначить клавиши и для перемещения пера по экрану использовать буквы нижнего регистра: “h”, “j”, “k” и “l”. Те же буквы, но верхнего регистра (“H”, “J”, “K” и “L”), перемещают перо на большее расстояние. Отслеживание состояния клавиш осуществляется с помощью метода `keyPressed()`, а получение символов — посредством метода `keyTyped()`.



Рис. 8.7. Программа рисования

В данной программе есть одна деталь: обычно панель не может получать никаких событий, связанных с нажатием клавиш. Для того чтобы обойти это ограничение, мы вызываем метод `setFocusable()`. Вопросы получения фокуса ввода мы рассмотрим далее в этой главе.

Листинг 8.3. Содержимое файла Sketch.java

```

import java.awt.*;
import java.awt.geom.*;
import java.util.*;
import java.awt.event.*;
import javax.swing.*;

public class Sketch
{
    public static void main(String[] args)
    {
        SketchFrame frame = new SketchFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм с панелью для рисования фигур.
 */
class SketchFrame extends JFrame
{
    public SketchFrame()
    {
        setTitle("Sketch");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        SketchPanel panel = new SketchPanel();
        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель для рисования с помощью клавиатуры.
 */
class SketchPanel extends JPanel
{
    public SketchPanel()
    {
        last = new Point2D.Double(100, 100);
        lines = new ArrayList<Line2D>();
        KeyHandler listener = new KeyHandler();
        addKeyListener(listener);
        setFocusable(true);
    }

    /**
     * Добавление нового отрезка к рисунку.
     * @param dx Перемещение в направлении x
     * @param dy Перемещение в направлении y
     */

```

```

*/
public void add(int dx, int dy)
{
    // Вычисление конца нового отрезка.
    Point2D end = new Point2D.Double(last.getX() + dx,
        last.getY() + dy);

    // Добавление нового отрезка.
    Line2D line = new Line2D.Double(last, end);
    lines.add(line);
    repaint();

    // Запоминается очередная конечная точка.
    last = end;
}

public void paintComponent(Graphics g)
{
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;

    // Рисование всех линий.
    for (Line2D l : lines)
        g2.draw(l);
}

private Point2D last;
private ArrayList<Line2D> lines;

private static final int SMALL_INCREMENT = 1;
private static final int LARGE_INCREMENT = 5;

private class KeyHandler implements KeyListener
{
    public void keyPressed(KeyEvent event)
    {
        int keyCode = event.getKeyCode();

        // Установка расстояния.
        int d;
        if (event.isShiftDown())
            d = LARGE_INCREMENT;
        else
            d = SMALL_INCREMENT;

        // Добавление нового отрезка.
        if (keyCode == KeyEvent.VK_LEFT) add(-d, 0);
        else if (keyCode == KeyEvent.VK_RIGHT) add(d, 0);
        else if (keyCode == KeyEvent.VK_UP) add(0, -d);
        else if (keyCode == KeyEvent.VK_DOWN) add(0, d);
    }

    public void keyReleased(KeyEvent event) {}

    public void keyTyped(KeyEvent event)
    {

```

```

char keyChar = event.getKeyChar();

// Установка расстояния.
int d;
if (Character.isUpperCase(keyChar))
{
    d = LARGE_INCREMENT;
    keyChar = Character.toLowerCase(keyChar);
}
else
    d = SMALL_INCREMENT;

// Добавление нового отрезка.
if (keyChar == 'h') add(-d, 0);
else if (keyChar == 'l') add(d, 0);
else if (keyChar == 'k') add(0, -d);
else if (keyChar == 'j') add(0, d);
    }
}
}

```



java.awt.event.KeyEvent 1.1

- **char getKeyChar()**
Возвращает символ, введенный пользователем.
- **int getKeyCode()**
Возвращает виртуальный код клавиши, соответствующий данному событию.
- **boolean isActionKey()**
Возвращает значение **true**, если данное событие связано с одной из следующих клавиш: <Home>, <End>, <Page Up>, <Page Down>, <Up>, <Down>, <Left>, <Right>, <F1 ... F24>, <Print Screen>, <Scroll Lock>, <Caps Lock>, <Num Lock>, <Pause>, <Insert>, <Delete>, <Enter>, <Backspace> и <Tab>.
- **static String getKeyText(int keyCode)**
Возвращает строку, описывающую клавишу. Например, вызов `getKeyText(KeyEvent.VK_END)` возвращает строку "END".
- **static String getKeyModifiersText(int modifiers)**
Возвращает строку, описывающую модификаторы, например <Shift> или <Ctrl>+<Shift>.
Параметр **modifiers** Состояние модификаторов



java.awt.event.InputEvent 1.1

- **int getModifiers()**
Возвращает целое число, описывающее состояние модификаторов <Shift>, <Ctrl> <Alt> и <Meta>. Этот метод применяется как для клавиатуры, так и для мыши. Для того чтобы проверить, установлен ли определенный разряд, при-

меняются маски `SHIFT_MASK`, `CTRL_MASK`, `META_MASK` или вызывается один из перечисленных ниже методов.

- `boolean isShiftDown()`
- `boolean isControlDown()`
- `boolean isAltDown()`
- `boolean isAltGraphDown()` 1.2
- `boolean isMetaDown()`

Эти методы возвращают значение `true`, если клавиша-модификатор в момент генерации данного события была нажата.

События, связанные с мышью

Если пользователь должен только щелкать на кнопках или выбирать пункты меню, обрабатывать явным образом события, связанные с мышью, не нужно. Эти операции автоматически поддерживаются компонентами пользовательского интерфейса, а затем преобразовываются в соответствующие семантические события. Однако, если пользователь должен иметь возможность рисовать с помощью мыши, необходимо отслеживать ее перемещения, щелчки и события, связанные с перетаскиванием объектов.

В данном разделе мы рассмотрим простой графический редактор, позволяющий создавать, перемещать и стирать квадраты на холсте (рис. 8.8).



Рис. 8.8. Программа, демонстрирующая обработку событий, связанных с мышью

Когда пользователь щелкает мышью, вызываются три метода объекта-обработчика: `mousePressed()`, если кнопка мыши была нажата, `mouseReleased()`, если кнопка была отпущена, и `mouseClicked`. Если надо отследить только сам щелчок, первые два метода использовать не обязательно. Пользуясь методами `getX()` и `getY()` объекта `MouseEvent`, передаваемого в качестве параметра, можно определить координаты курсора мыши в момент щелчка. Если нужно различать обычный, двойной и тройной (!) щелчки, используется метод `getClickCount()`.

Некоторые разработчики так реализуют свое программное обеспечение, что для управления интерфейсными элементами пользователю приходится щелкать мышью при нажатых клавишах. В исходной библиотеке API маски, соответствующие двум кнопкам мыши, совпадали с масками клавиш-модификаторов:

```
BUTTON2_MASK == ALT_MASK
BUTTON3_MASK == META_MASK
```

Вероятно, это было сделано для того, чтобы пользователи однокнопочных мышей могли имитировать нажатие второй кнопки с помощью нажатия клавиши на клавиша-

туре. Однако в пакете JDK 1.4 использован другой подход. Теперь в нем предусмотрено несколько масок.

```

BUTTON1_DOWN_MASK
BUTTON2_DOWN_MASK
BUTTON3_DOWN_MASK
SHIFT_DOWN_MASK
CTRL_DOWN_MASK
ALT_DOWN_MASK
ALT_GRAPH_DOWN_MASK
META_DOWN_MASK

```

Метод `getModifiersEx()` позволяет распознавать события, связанные с нажатием кнопок мыши при нажатых клавишах.

Обратите внимание на то, что маска `BUTTON3_DOWN_MASK` позволяет проверять нажатие правой кнопки мыши в среде Windows. Например, для определения, нажата ли правая клавиша, можно воспользоваться следующим кодом:

```

if ((event.getModifiersEx() & InputEvent.BUTTON3_DOWN_MASK) != 0)
    // Код, обрабатывающий нажатие правой кнопки

```

В программе, которая будет рассмотрена ниже, используются методы `mousePressed()` и `mouseClicked()`. Если щелкнуть кнопкой мыши на пикселе, не принадлежащем ни одному нарисованному квадрату, на экране появится новый квадрат. Эта процедура реализована в методе `mousePressed()`, поэтому компьютер отреагирует на щелчок мыши немедленно, не дожидаясь освобождения кнопки мыши. Двойной щелчок внутри какого-нибудь квадрата приведет к тому, что он будет стерт. Эта процедура реализована в методе `mouseClicked()`, поскольку щелчки мыши нужно подсчитывать.

```

public void mousePressed(MouseEvent event)
{
    current = find(event.getPoint());
    if (current == null) // Вне квадрата.
        add(event.getPoint());
}

public void mouseClicked(MouseEvent event)
{
    current = find(event.getPoint());
    if (current != null && event.getClickCount() >= 2)
        remove(current);
}

```

При перемещении мыши по окну последнее получает постоянный поток событий, связанных с движением мыши. В большинстве приложений эти события игнорируются. Однако в нашем примере программа отслеживает события, чтобы изменить вид курсора (на крестообразный), как только он выйдет за пределы квадрата. Эта процедура реализована в методе `getPredefinedCursor()` в классе `Cursor`. В табл. 8.2 приведены константы, используемые в этом методе, и показаны виды курсоров, предусмотренные в среде Windows.

Таблица 8.2. Виды курсоров

Пиктограмма	Константа
	DEFAULT_CURSOR
	CROSSHAIR_CURSOR
	HAND_CURSOR
	MOVE_CURSOR
	TEXT_CURSOR
	WAIT_CURSOR
	N_RESIZE_CURSOR
	NE_RESIZE_CURSOR
	E_RESIZE_CURSOR
	SE_RESIZE_CURSOR
	S_RESIZE_CURSOR
	SW_RESIZE_CURSOR
	W_RESIZE_CURSOR
	NW_RESIZE_CURSOR

(Обратите внимание на то, что некоторые виды курсоров выглядят одинаково. На других платформах они, возможно, будут различаться.)



Изображения курсоров можно найти в каталоге `jre/lib/images/cursors`. В файле `cursor.properties` определены точки, являющиеся указателями. Например, если курсор имеет вид руки, то указателем считается кончик указательного пальца. Если курсор имеет вид лупы, то указателем является центр этой лупы.

Рассмотрим метод `mouseMoved()`, объявленный в интерфейсе `MouseMotionListener` и реализованный в нашей программе.

```
public void mouseMoved(MouseEvent event)
{
    if (find(event.getPoint()) == null)
        setCursor(Cursor.getDefaultCursor());
    else
        setCursor(Cursor.getPredefinedCursor(
            Cursor.CROSSHAIR_CURSOR));
}
```



Используя метод `createCustomCursor()` из класса `Toolkit`, можно определить новую форму курсора.

```
Toolkit tk = Toolkit.getDefaultToolkit();
Image img = tk.getImage("dynamite.gif");
Cursor dynamiteCursor = tk.createCustomCursor(img,
    new Point(10, 10), "dynamite stick");
```

Первый параметр метода `createCustomCursor()` задает изображение курсора. Второй параметр определяет координаты точки, используемой в качестве указателя. Третий параметр — строка, описывающая курсор. Эту строку можно использовать для вспомогательных целей, например, сообщить голосом вид курсора. Такая возможность важна при адаптации программ для пользователей с нарушениями зрения.

Если перемещение мыши осуществляется при нажатой кнопке, вместо `mouseClicked()` вызывается метод `mouseDragged()`. В нашем примере квадраты можно перетаскивать по экрану. При этом квадрат перемещается так, чтобы его центр располагался в той точке, на которую указывает мышь. Содержимое холста перерисовывается.

```
public void mouseDragged(MouseEvent event)
{
    if (cursor != null)
    {
        int x = event.getX();
        int y = event.getY();

        current.setFrame(
            x - SIDELENGTH / 2,
            y - SIDELENGTH / 2,
            SIDELENGTH,
            SIDELENGTH);
        repaint();
    }
}
```



Метод `mouseMoved()` вызывается, только если указатель мыши находится внутри компонента. Однако метод `mouseDragged()` вызывается даже тогда, когда указатель мыши пребывает вне компонента.

Есть еще два метода, обрабатывающих события, связанные с мышью: `mouseEntered()` и `mouseExited()`. Эти методы вызываются, когда указатель мыши входит в пределы компонента и выходит из него.

Заметим еще одну особенность, связанную с обработкой событий мыши. В ответ на щелчок мышью вызывается метод `mouseClicked()`, являющийся частью интерфейса `MouseListener`. Поскольку во многих приложениях отслеживаются только щелчки мышью, а также потому, что перемещения мыши происходят слишком часто, события, связанные с перемещением мыши и перетаскиванием, определяются в отдельном интерфейсе `MouseMotionListener`.

В нашей программе нас интересуют оба типа событий, связанных с мышью. В ней определены два внутренних класса: `MouseHandler` и `MouseMotionHandler`. Класс `MouseHandler` расширяет класс `MouseAdapter`, поскольку он определяет только два из пяти методов класса `MouseListener`. Класс `MouseMotionHandler` реализует интерфейс `MouseMotionListener` и определяет оба его метода. Текст программы приведен в листинге 8.4.

Листинг 8.4. Содержимое файла `MouseTest.java`

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.awt.geom.*;
import javax.swing.*;

public class MouseTest
{
    public static void main(String[] args)
    {
        MouseFrame frame = new MouseFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм, содержащий панель, предназначенную для проверки
 * операций с мышью.
 */
class MouseFrame extends JFrame
{
    public MouseFrame()
    {
        setTitle("MouseTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        MousePanel panel = new MousePanel();
        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
```

```

}

/**
 * Панель, поддерживающая добавление и удаление квадратов
 * с помощью мыши.
 */
class MousePanel extends JPanel
{
    public MousePanel()
    {
        squares = new ArrayList<Rectangle2D>();
        current = null;

        addMouseListener(new MouseHandler());
        addMouseMotionListener(new MouseMotionHandler());
    }

    public void paintComponent(Graphics g)
    {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;

        // Рисование всех квадратов.
        for (Rectangle2D r : squares)
            g2.draw(r);
    }

    /**
     * Нахождение первого квадрата, который содержит заданную точку.
     * @param p Точка
     * @return Первый квадрат, содержащий точку p
     */
    public Rectangle2D find(Point2D p)
    {
        for (Rectangle2D r : squares)
        {
            if (r.contains(p)) return r;
        }
        return null;
    }

    /**
     * Добавление квадрата к набору.
     * @param p Центр квадрата
     */
    public void add(Point2D p)
    {
        double x = p.getX();
        double y = p.getY();

        current = new Rectangle2D.Double(
            x - SIDELENGTH / 2,
            y - SIDELENGTH / 2,
            SIDELENGTH,
            SIDELENGTH);
        squares.add(current);
    }
}

```

```

        repaint();
    }

    /**
     * Удаление квадрата из набора.
     * @param s Удаляемый квадрат
     */
    public void remove(Rectangle2D s)
    {
        if (s == null) return;
        if (s == current) current = null;
        squares.remove(s);
        repaint();
    }

    private static final int SIDELENGTH = 10;
    private ArrayList<Rectangle2D> squares;
    private Rectangle2D current;
    // Квадрат, в пределах которого находится курсор.

    private class MouseHandler extends MouseAdapter
    {
        public void mousePressed(MouseEvent event)
        {
            // Если курсор находится за пределами всех квадратов,
            // добавляется новый квадрат
            current = find(event.getPoint());
            if (current == null)
                add(event.getPoint());
        }

        public void mouseClicked(MouseEvent event)
        {
            // После двойного щелчка текущий квадрат удаляется.
            current = find(event.getPoint());
            if (current != null && event.getClickCount() >= 2)
                remove(current);
        }
    }

    private class MouseMotionHandler
        implements MouseMotionListener
    {
        public void mouseMoved(MouseEvent event)
        {
            // Если курсор находится в пределах квадрата,
            // он принимает крестообразный вид.

            if (find(event.getPoint()) == null)
                setCursor(Cursor.getDefaultCursor());
            else
                setCursor(Cursor.getPredefinedCursor(Cursor.CROSSHAIR_CURSOR));
        }
    }

```

```

public void mouseDragged(MouseEvent event)
{
    if (current != null)
    {
        int x = event.getX();
        int y = event.getY();

        // Перетаскивание текущего квадрата в точку (x, y)
        current.setFrame(
            x - SIDELENGTH / 2,
            y - SIDELENGTH / 2,
            SIDELENGTH,
            SIDELENGTH);
        repaint();
    }
}
}

```



java.awt.event.MouseEvent 1.1

- int getX()
- int getY()
- Point getPoint()

Возвращает горизонтальную (x) и вертикальную (y) координаты или точку, соответствующую событию. Отсчет производится от левого верхнего угла компонента, являющегося источником события.

- void translatePoint(int x, int y)

Преобразует координаты события, перемещая точку на x единиц горизонтально и y единиц вертикально.

- int getClickCount()

Возвращает количество последовательных щелчков мыши, связанных с данным событием. (Интервал времени, в пределах которого подсчитываются щелчки, зависит от операционной системы.)



java.awt.event.InputEvent 1.1

- int getModifiersEx() 1.4

Возвращает модификаторы для события. При проверке возвращаемого значения используются перечисленные ниже маски.

```

BUTTON1_DOWN_MASK
BUTTON2_DOWN_MASK
BUTTON3_DOWN_MASK
SHIFT_DOWN_MASK
CTRL_DOWN_MASK
ALT_DOWN_MASK
ALT_GRAPH_DOWN_MASK
META_DOWN_MASK

```

- `static int getModifiersExText(int modifiers)` **1.4**

Возвращает строку, описывающую модификаторы, например Shift+Button1.



java.awt.Toolkit 1.0

- `public Cursor createCustomCursor(Image image, Point hotSpot, String name)` **1.2**

Создает новый объект курсора.

Параметры	image	Изображение активного курсора
	hotSpot	Точка, считающаяся указателем
	name	Описание курсора



java.awt.Component 1.0

- `public void setCursor(Cursor cursor)` **1.1**

Изменяет внешний вид курсора.

События, связанные с фокусом ввода

Пользуясь мышью, можно указать любой объект на экране. Однако при вводе текста нажатие клавиши должно передаваться конкретному объекту на экране. *Оконный диспетчер* направляет все нажатия клавиш *активному окну* (active window). Обычно активное окно отличается от остальных цветом строки заголовка. Активным может быть лишь одно окно.

Предположим, что активным окном управляет программа, написанная на языке Java. Это окно получает информацию о нажатии клавиш и направляет ее конкретному *компоненту*. В таком случае говорят, что этот компонент обладает фокусом ввода. Аналогично тому, как активное окно выделяется среди остальных цветом строки заголовка, компонент, имеющий фокус, визуально отличается от других. В поле ввода текста мигает курсор, активная кнопка выделяется прямоугольником, обрамляющим ее заголовок, и т.д. Если фокус принадлежит полю редактирования, в нем можно вводить текст. Если фокусом обладает кнопка, на ней можно щелкнуть, нажав клавишу пробела.

В каждый момент времени фокус ввода может принадлежать только одному компоненту. Компонент может *потерять фокус ввода*, если пользователь выбрал другой компонент. Этому компоненту передается фокус ввода. Компонент получает фокус, если пользователь щелкнул на нем мышью. Для этой цели можно воспользоваться также клавишей <Tab>, которая перемещает фокус с одного компонента на другой. Таким образом, можно пройти по всем компонентам, которые потенциально могут обладать фокусом. По умолчанию перемещение по компонентам библиотеки Swing осуществляется слева направо, а затем сверху вниз в соответствии с их расположением в контейнере. Порядок обхода компонентов можно изменить (подробная информация об этом приведена в следующей главе).

К счастью, большинство прикладных программистов не слишком беспокоятся об обработке событий, связанных с фокусом. До появления пакета JDK 1.4 этот механизм обычно использовался для проверки возможных ошибок или верификации данных. Допустим, что в текстовое поле вводится номер кредитной карточки и после завершения редактирования пользователь переходит в новое поле. В этот момент можно

перехватить событие, связанное с потерей фокуса. Если номер кредитной карточки не соответствует ожидаемому формату, можно вывести сообщение об ошибке и вернуть фокус предыдущему полю ввода. Однако в пакете JDK 1.4 есть более простые и надежные механизмы верификации. Мы обсудим их в главе 9.



К сожалению, в ранних версиях пакета JDK обработка событий, связанных с фокусом ввода, была реализована не слишком удачно. Пользователи обнаружили больше сотни разных ошибок. Это явление объясняется двумя причинами. Во-первых, фокус компонента взаимодействует с фокусом окна, который находится в компетенции оконного диспетчера. Следовательно, поведение фокуса зависит от платформы, на которой выполняется приложение. Во-вторых, реализация кода, очевидно, быстро вышла из-под контроля, особенно в сочетании с неудовлетворительной реализацией механизма отслеживания фокуса в пакете JDK 1.3.

Уинстон Черчилль (Winston Churchill) однажды сказал: “Американцы всегда поступают правильно... после того как исчерпают все остальные возможности”. Очевидно, то же самое относится и к группе разработчиков языка Java. Они все же реализовали правильный механизм отслеживания фокусов в пакете JDK 1.4. В этом пакете полностью переработан код обработки фокуса и подробно описано его ожидаемое поведение, включая неизбежную зависимость от платформы.

Спецификацию механизма обработки фокуса ввода можно найти на Web-странице <http://java.sun.com/j2se/~1.4/docs/api/java/awt/doc-files/FocusSpec.html>.

Некоторые компоненты, например метки и панели, по умолчанию не получают фокус ввода, поскольку предполагается, что они используются для отображения сообщений или объединения нескольких компонентов в одно целое. Если вы хотите написать приложение, которое позволяло бы рисовать на панели, нажимая клавиши, то необходимо сделать так, чтобы панель могла обладать фокусом. Работая с JDK 1.4, для этого достаточно вызвать метод `setFocusable()`.

```
panel.setFocusable(true);
```



В старых версиях JDK переопределение метода `isFocusTraversable()`, принадлежащего компоненту, приводило к такому же результату. Однако в старых пакетах использовалась другая концепция фокуса ввода. Это различие вводило пользователей в заблуждение, поэтому метод `isFocusTraversable()` был объявлен как не рекомендованный к применению.

В оставшейся части раздела мы обсудим детали обработки событий, связанных с фокусом ввода. Этот текст вы можете спокойно пропустить, пока не почувствуете необходимость использовать описанный механизм в своих приложениях.

В пакете JDK 1.4 используются следующие понятия.

- *Владелец фокуса* (focus owner), т.е. компонент, обладающий фокусом ввода.
- Окно, содержащее владельца фокуса.
- *Активное окно* (active window), т.е. фрейм или диалоговое окно, содержащее владельца фокуса.

Окно, содержащее владельца фокуса, обычно является активным. Однако возможна ситуация, когда владелец фокуса содержится в окне верхнего уровня, не являющемся фреймом, например во всплывающем меню.

Чтобы получить эту информацию о владельце, об окне, содержащем его, и активном окне, сначала надо получить диспетчер фокуса клавиатуры.

```
KeyboardFocusManager manager =
    KeyboardFocusManager.getCurrentKeyboardFocusManager();
```

Затем следует вызвать приведенные ниже методы.

```
Component owner = manager.getFocusOwner();
Window focused = manager.getFocusedWindow();
Window active = manager.getActiveWindow(); // Фрейм или диалоговое окно
```

Для регистрации перемещений фокуса ввода необходимо связать обработчики событий с компонентами или окнами. Обработчик должен реализовывать два метода: `focusGained()` и `focusLost()`. Эти методы вызываются тогда, когда *источник* события получает фокус ввода или теряет фокус. Каждому из этих методов передается параметр `FocusEvent`. В этом классе также есть два полезных метода: метод `getComponent()` сообщает, обладает ли компонент фокусом ввода, а метод `isTemporary()` возвращает значение `true`, если фокус ввода передан временно. Временная передача фокуса происходит тогда, когда компонент теряет управление, но впоследствии автоматически получает его обратно. Например, если пользователь переходит к другому окну, а затем возвращается к предыдущему, исходный компонент снова получает фокус ввода.

В JDK 1.4 реализован механизм доставки событий, связанных с фокусом ввода для окна. Для этого с окном связывается класс `WindowFocusListener` и переопределяются методы `windowGainedFocus()` и `windowLostFocus()`.

Кроме того, в пакете JDK 1.4 введено понятие “дополняющего”, или “противоположного”, компонента или окна. Когда компонент или окно теряют фокус ввода, “противоположный” компонент или окно получают фокус. И наоборот, когда исходный компонент или окно вновь получает фокус, их “противоположное” окно теряет его. Метод `getOppositeComponent()` класса `FocusEvent` возвращает “противоположный” компонент, а метод `getOppositeWindow()` класса `WindowEvent` — “противоположное” окно.

Фокус ввода можно запрограммировать с помощью метода `requestComponent()` класса `Component`. Однако поведение фокуса существенно зависит от платформы, на которой выполняется приложение. Для создания платформенно-независимых программ в пакете JDK 1.4 предусмотрен метод `requestFocusInWindow()` класса `Component`. Этот метод работает, только если компонент содержится внутри окна.



Не следует считать, что ваш компонент обладает фокусом ввода лишь на основании того, что метод `requestFocus()` или `requestFocusInWindow()` вернул значение `true`. Лучше дождаться события `FOCUS_GAINED`.



Некоторые программисты неверно понимают суть события `FOCUS_LOST` и пытаются предотвратить получение фокуса другим компонентом, вызывая обработчик `focusLost`. Однако к этому времени компонент уже потерял фокус ввода. Если вам

необходимо отслеживать перехват фокуса конкретным компонентом, необходимо установить обработчик, обладающий правом вето и запретить изменения свойства `focusOwner`. Подробно данный механизм описан в главе 8 тома 2.



java.awt.Component 1.0

- `void requestFocus()`
Запрашивает передачу фокуса данному компоненту.
- `Boolean requestFocusInWindow()` 1.4
Запрашивает передачу фокуса данному компоненту. Возвращает значение `false`, если данный компонент не содержится в окне или запрос невозможно выполнить по другой причине. Возвращает значение `true`, если запрос можно выполнить.
- `void setFocusable(boolean b)` 1.4
- `boolean isFocusable()` 1.4
Устанавливает или определяет способность компонента обладать фокусом ввода. Если параметр `b` имеет значение `true`, значит, компонент может получать фокус.
- `boolean isFocusOwner()` 1.4
Возвращает значение `true`, если компонент в данный момент обладает фокусом ввода.



java.awt.KeyboardFocusManager 1.4

- `static KeyboardFocusManager getCurrentKeyboardFocusManager()`
Определяет текущий диспетчер фокуса.
- `Component getFocusOwner()`
Определяет компонент, владеющий фокусом ввода, или возвращает ссылку `null`, если компонент, который обладает фокусом, не управляется диспетчером.
- `Window getFocusWindow()`
Определяет окно, которое содержит компонент, владеющий фокусом ввода, или возвращает ссылку `null`, если компонент, который обладает фокусом, не управляется диспетчером.
- `Window getActiveWindow()`
Определяет диалоговое окно или фрейм, содержащее окно, в котором находится компонент, владеющий фокусом ввода. Если окно не управляется диспетчером, возвращает ссылку `null`.



java.awt.Window 1.0

- `boolean isFocused()` 1.4
Возвращает значение `true`, если окно содержит элемент, обладающий фокусом ввода.

- `boolean isActive()` 1.4

Возвращает значение `true`, если фрейм или диалоговое окно является активным. Строка заголовка активного фрейма или диалогового окна обычно выделяется цветом.



`java.awt.event.FocusEvent` 1.1

- `Component getOppositeComponent()` 1.4

Возвращает компонент, который потерял фокус в обработчике `focusGained` или получил фокус в обработчике `focusLost`.



`java.awt.event.WindowEvent` 1.4

- `Component getOppositeWindow()` 1.4

Возвращает окно, потерявшее фокус в обработчике `windowGainedFocus`, либо окно, которое получило фокус в обработчике `windowLostFocus`, либо окно, активизированное в обработчике `windowActivated`, или окно, деактивизированное в обработчике `windowDeactivated`.



`java.awt.event.WindowFocusListener` 1.4

- `void windowGainedFocus(WindowEvent event)`

Вызывается, когда окно, являющееся источником события, получает фокус ввода.

- `void windowLostFocus(WindowEvent event)`

Вызывается, когда окно, являющееся источником события, теряет фокус ввода.

Действия

Одну и ту же команду можно выполнить разными способами. Пользователь может выбрать пункт меню, нажать соответствующую клавишу или щелкнуть на кнопке панели инструментов. В этом случае очень удобна рассматриваемая нами модель событий: достаточно связать все эти события с одним и тем же обработчиком. Допустим, что `blueAction` — это объект некоего класса (скажем, `ColorAction`), реализующего интерфейс `ActionListener` и изменяющего цвет фона на синий. Можно связать один и тот же объект с разными источниками событий, перечисленными ниже.

- Кнопка `Blue` на панели инструментов.
- Пункт меню `Blue`.
- Нажатие клавиш `<Ctrl+B>`.

Команда, изменяющая цвет, выполняется одинаково, независимо от того, что привело к ее выполнению — щелчок на кнопке, выбор пункта меню или нажатие клавиш.

В пакете `Swing` предусмотрен очень полезный механизм, позволяющий инкапсулировать команды и связывать их с несколькими источниками событий. Этот механизм представляет собой интерфейс `Action`. *Действие* (action) — это объект, инкапсулирующий описание команды (в виде текстовой строки или пиктограммы) и параметры, необходимые для выполнения программы (например, нужный цвет).

Интерфейс `Action` содержит следующие методы:

```
void actionPerformed(ActionEvent event)
void setEnabled(boolean b)
boolean isEnabled()
void putValue(String key, Object value)
Object getValue(String key)
void addPropertyChangeListener(PropertyChangeListener listener)
void removePropertyChangeListener(PropertyChangeListener listener)
```

Первый метод похож на метод интерфейса `ActionListener`: действительно, интерфейс `Action` расширяет интерфейс `ActionListener`. Следовательно, вместо объекта класса, реализующего интерфейс `ActionListener`, можно использовать объект класса, реализующего интерфейс `Action`.

Следующие два метода позволяют блокировать и разблокировать действие, а также проверить, является ли указанное действие разблокированным (т.е. доступным) в данный момент. Если пункт меню или кнопка панели инструментов связаны с заблокированным действием, они выделяются светло-серым цветом.

Методы `putValue()` `getValue()` позволяют записывать и извлекать из памяти произвольные пары имя–значение объектов класса, реализующего интерфейс `Action`. Предопределенные строки, такие как `Action.NAME` и `Action.SMALL_ICON`, содержат имена и пиктограммы объектов действий.

```
Action.putValue(Action.NAME, "Blue");
action.putValue(Action.SMALL_ICON,
    new ImageIcon("blue-ball.gif"));
```

В табл. 8.3 приведены предопределенные имена.

Таблица 8.3. Заранее определенные имена действий

Имя	Значение
<code>NAME</code>	Имя действия. Отображается на кнопке и в названии пункта меню
<code>SMALL_ICON</code>	Место для хранения пиктограммы, которая изображается на кнопке, в пункте меню или на панели инструментов
<code>SHORT_DESCRIPTION</code>	Краткое описание пиктограммы, отображается в строке подсказки
<code>LONG_DESCRIPTION</code>	Подробное описание пиктограммы. Может использоваться для подсказки. Не используется ни одним компонентом из библиотеки Swing
<code>MNEMONIC_KEY</code>	Мнемоническое сокращение. Отображается в пункте меню (см. главу 9)
<code>ACCELERATOR_KEY</code>	Место для хранения сочетания клавиш. Не используется ни одним компонентом из библиотеки Swing
<code>ACTION_COMMAND_KEY</code>	Ранее использовалось в теперь уже устаревшем методе <code>registeredKeyboardAction()</code>
<code>DEFAULT</code>	Может быть полезным для хранения разнообразных объектов. Не используется ни одним компонентом из библиотеки Swing

Если в меню или к панели инструментов добавляется какое-то действие, его имя и пиктограмма автоматически извлекаются из памяти и отображаются в меню и на панели. Значение `SHORT_DESCRIPTION` выводится в строке подсказки.

Последние два метода интерфейса `Action` позволяют извещать другие объекты, в частности меню и панели инструментов, об изменении свойств действий. Например, если меню создано как обработчик, предназначенный для отслеживания изменения свойств действия, и это действие впоследствии было заблокировано, то при отображении меню на экране соответствующее имя действия может быть выделено серым цветом. Обработчики изменения свойств представляют собой часть модели `JavaBeans`. Подробно компоненты `JavaBeans` и их свойства будут рассматриваться во втором томе.

Обратите внимание на то, что `Action` является *интерфейсом*, а не классом. Любой класс, реализующий этот интерфейс, должен реализовать семь методов, которые мы только что рассмотрели. К счастью, это сделать достаточно легко, поскольку все они, кроме первого метода, содержатся в классе `AbstractAction`. Класс `AbstractAction` предназначен для хранения пар, состоящих из имен и значений, а также для управления обработчиками изменения свойств. Все, что нужно для этого сделать для практического использования, — это создать подкласс и добавить метод `actionPerformed()`.

Попробуем создать действие, изменяющее цвет фона. Поместим в память имя команды, соответствующую пиктограмму и желаемый цвет. Код цвета запишем в таблицу, состоящую из пар имя–значение, предусмотренных классом `AbstractAction`. Ниже приводится класс `ColorAction`. Конструктор задает пары, состоящие из имен и значений, а метод `ActionPerformed()` изменяет цвет фона.

```
public class ColorAction extends AbstractAction
{
    public ColorAction(String name, Icon icon, Color c)
    {
        putValue(Action.NAME, name);
        putValue(Action.SMALL_ICON, icon);
        putValue("color", c);
        putValue(Action.SHORT_DESCRIPTION,
            "Set panel color to " + name.toLowerCase());
    }

    public void actionPerformed(ActionEvent event)
    {
        Color c = (Color) getValue("color");
        setBackground(c);
    }
}
```

В программе создаются три объекта этого класса.

```
Action blueAction = new ColorAction("Blue",
    new ImageIcon("blue-ball.gif"), Color.BLUE);
```

Теперь свяжем это действие с кнопкой. Сделать это можно с помощью конструктора `JButton`, получающего объект `Action` в качестве параметра.

```
JButton blueButton = new JButton(blueAction);
```

Этот конструктор считывает имя и пиктограмму действия, помещает короткое описание в строку подсказки и регистрирует объект `Action` в качестве обработчика. Пиктограмма и строка подсказки показаны на рис. 8.9.

Как мы увидим в следующей главе, точно так же легко можно добавлять действия в меню.



Рис. 8.9. Кнопки с пиктограммами из объекта класса `ColorAction`, реализующего интерфейс `Action`

Кроме того, нужно связать действия с нажатием клавиш. Для этого необходимо на время погрузиться в технические детали. Нажатия кнопок передаются компонентам, обладающим фокусом ввода. В нашем примере предусмотрены три компонента — три кнопки на панели. Следовательно, в любой момент одна из этих трех кнопок может владеть фокусом. Каждая из этих кнопок должна обрабатывать события, связанные с нажатием клавиш, и реагировать на сочетания клавиш `<Ctrl+Y>`, `<Ctrl+B>` и `<Ctrl+R>`.

Эта проблема имеет универсальный характер, и разработчики пакета `Swing` нашли ее удачное решение.



На самом деле в версии 1.2 пакета `JDK` были предложены два решения, позволяющих связать клавиши с действиями: метод `registerKeyboardAction()` класса `JComponent` и механизм отображения клавиш для команд `JTextComponent`. В версии `JDK 1.3` решения были объединены. В данном разделе описывается именно этот унифицированный подход.

Для того чтобы связать действия с нажатием клавиш, сначала нужно создать объект класса `KeyStroke`. Это удобный класс, инкапсулирующий описание клавиш. Чтобы создать объект этого класса, не нужно вызывать конструктор. Вместо него следует использовать статический метод `getKeyStroke()` класса `KeyStroke`. Теперь необходимо указать виртуальный код клавиш и флаги (например, для сочетаний, в состав которых входят клавиши `<Shift>` или `<Ctrl>`).

```
KeyStroke ctrlBKey = KeyStroke.getKeyStroke(KeyEvent.VK_B,
    InputEvent.CTRL_MASK);
```

Существует также способ, позволяющий описывать нажатие клавиши в виде строки.

```
KeyStroke ctrlBKey = KeyStroke.getKeyStroke("ctrl B");
```

Каждый объект класса `JComponent` содержит три *отображения ввода* (input map), связывающих объекты класса `KeyStroke` с действиями. Эти отображения соответствуют различным условиям (табл. 8.4).

Таблица 8.4. Условия, соответствующие трем отображениям ввода

Флаг	Условие
<code>WHEN_FOCUSED</code>	Когда данный компонент находится в фокусе клавиатуры
<code>WHEN_ANCESTOR_OF_FOCUSED_COMPONENT</code>	Когда данный компонент содержит другой компонент, находящийся в фокусе клавиатуры
<code>WHEN_IN_FOCUSED_WINDOW</code>	Когда данный компонент содержится в том же окне, что и компонент, находящийся в фокусе клавиатуры

При нажатии клавиши отображения ввода обрабатываются в следующем порядке.

1. Проверяется отображение `WHEN_FOCUSED` компонента, владеющего фокусом ввода. Если предусмотрена реакция на нажатие клавиши, выполняется соответствующее действие, если действие не заблокировано, процесс прекращается.
2. Начиная с компонента, содержащего фокус ввода, выполняется проверка условия `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` его родительского компонента. Как только условие будет выполнено, производится соответствующее действие. Если действие не заблокировано, процесс прекращается.
3. Проверяются все видимые и незаблокированные компоненты в окне, обладающем фокусом ввода. Компоненты должны быть зарегистрированы в условии `WHEN_IN_FOCUSED_WINDOW`. Каждый из этих компонентов (в порядке регистрации) получает возможность выполнить соответствующее действие. Когда будет выполнено первое незаблокированное действие, процесс прекратится. Если клавиша присутствует в нескольких отображениях `WHEN_IN_FOCUSED_WINDOW`, то результаты, полученные на данном этапе, могут быть неоднозначными.

Отображение ввода извлекается из компонента с помощью метода `getInputMap()`.

```
InputMap imap = panel.getInputMap(JComponent.WHEN_FOCUSED);
```

Условие `WHEN_FOCUSED` означает, что данное отображение проверяется, если компонент обладает фокусом ввода. В нашем случае это условие не проверяется, поскольку фокусом владеет одна кнопка, а не панель в целом. Каждое из оставшихся двух отображений также позволяет очень легко изменить цвет фона в ответ на нажатие клавиш. В программе мы будем использовать `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT`.

Класс `InputMap` не связывает напрямую объекты класса `KeyStroke` с объектами класса `ColorAction`, реализующего интерфейс `Action`. Вместо этого объектам класса `KeyStroke` он ставит в соответствие произвольные объекты, а с помощью второго отображения, реализованного классом `ActionMap`, связывает их с действиями. Это облегчает связывание одного и того же действия с разными клавишами, зарегистрированными в разных отображениях ввода.

Итак, каждый компонент имеет три отображения ввода и одно отображение, связывающее нажатие клавиш с действиями. Чтобы объединить их, нужно присвоить каждому действию имя.

```
imap.put(KeyStroke.getKeyStroke("ctrl Y"), "panel.yellow");
ActionMap amap = panel.getActionMap();
amap.put("panel.yellow", yellowAction);
```

Если нужно задать “пустое” действие, можно использовать строку `"none"`. Это позволяет легко заблокировать клавишу.

```
imap.put(KeyStroke.getKeyStroke("ctrl C", "none");
```



В документации JDK предполагается, что название клавиш и соответствующего действия совпадают. Такое решение вряд ли можно назвать оптимальным. Имя действия отображается на кнопке и в пункте меню. Оно может изменяться в процессе разработки, в частности, это неизбежно при адаптации пользовательского интерфейса для разных языков. Мы рекомендуем делать имена действий, которые выводятся на экран, не зависящими от их настоящих имен.

Итак, чтобы одна и та же операция выполнялась в ответ на щелчок на кнопке, выбор пункта меню или нажатие клавиши, надо сделать следующее

1. Создать класс, расширяющий класс `AbstractAction`. Один класс можно будет использовать для программирования разных действий.
2. Создать объект класса, реализующего интерфейс `Action`.
3. Создать кнопку или пункт меню на основе объекта класса, реализующего интерфейс `Action`.
4. Для действий, которые выполняются в ответ на нажатие клавиш, нужно сделать один дополнительный шаг. Сначала следует идентифицировать компонент окна верхнего уровня, например панель, содержащую все остальные компоненты.
5. Проверить отображение `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` компонента верхнего уровня. Создать объект класса `KeyStroke` для нужного нажатия клавиш. Создать объект, соответствующий нажатию клавиш, например строку, описывающую нужное действие. Добавить пару (нажатие клавиши, действие) к отображению ввода.
6. Получить информацию о соответствии действий и клавиш для компонента верхнего уровня. Добавить пару (клавиша, действие) в отображение.

В листинге 8.5 приведен полный текст программы, задающей соответствие между кнопками, нажатием клавиш и действиями. Действия выполняются в ответ на активизацию кнопок или нажатие клавиш `<Ctrl+Y>`, `<Ctrl+B>` или `<Ctrl+R>`.

Листинг 8.5. Содержимое файла `ActionTest.java`

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ActionTest
{
    public static void main(String[] args)
    {
        ActionFrame frame = new ActionFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм с панелью, демонстрирующий изменение цвета.
 */
class ActionFrame extends JFrame
{
    public ActionFrame()
    {
        setTitle("ActionTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        ActionPanel panel = new ActionPanel();
```

```

        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель, цвет которой можно изменять щелчком на кнопке или
 * нажатием клавиш.
 */
class ActionPanel extends JPanel
{
    public ActionPanel()
    {
        // Определение действий.

        Action yellowAction = new ColorAction("Yellow",
            new ImageIcon("yellow-ball.gif"),
            Color.YELLOW);
        Action blueAction = new ColorAction("Blue",
            new ImageIcon("blue-ball.gif"),
            Color.BLUE);
        Action redAction = new ColorAction("Red",
            new ImageIcon("red-ball.gif"),
            Color.RED);

        // Кнопки, предназначенные для выполнения операций.
        add(new JButton(yellowAction));
        add(new JButton(blueAction));
        add(new JButton(redAction));

        // Связывание клавиш Y, B и R с именами.
        InputMap imap =
            getInputMap(JComponent.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT);

        imap.put(KeyStroke.getKeyStroke("ctrl Y"), "panel.yellow");
        imap.put(KeyStroke.getKeyStroke("ctrl B"), "panel.blue");
        imap.put(KeyStroke.getKeyStroke("ctrl R"), "panel.red");

        // Связывание имен с действиями
        ActionMap amap = getActionMap();
        amap.put("panel.yellow", yellowAction);
        amap.put("panel.blue", blueAction);
        amap.put("panel.red", redAction);
    }

    public class ColorAction extends AbstractAction
    {
        /**
         * Создание действий для изменения цвета.
         * @param name Надпись на кнопке
         * @param icon Пиктограмма на кнопке
         * @param c Цвет фона

```

```

*/
public ColorAction(String name, Icon icon, Color c)
{
    putValue(Action.NAME, name);
    putValue(Action.SMALL_ICON, icon);
    putValue(Action.SHORT_DESCRIPTION,
        "Set panel color to " + name.toLowerCase());
    putValue("color", c);
}

public void actionPerformed(ActionEvent event)
{
    Color c = (Color) getValue("color");
    setBackground(c);
}
}
}

```



javax.swing.Action 1.2

- void setEnabled(boolean b)
Блокирует и разблокирует действия.
- boolean isEnabled()
Возвращает значение true, если действие разблокировано.
- void putValue(String key, Object value)
Помещает пару (имя, значение) в объект действия.

Параметры

key	Имя, которое должно сохраняться в объекте действия. Оно может быть строкой, но несколько имен имеют специальное значение
value	Объект, связанный с именем

- Object getValue(String key)
Возвращает значение из сохраненной пары (имя, значение).



javax.swing.JMenu 1.2

- JMenuItem add(Action a)
Добавляет в меню пункт, вызывающий данное действие. Возвращает добавленный пункт меню.



javax.swing.KeyStroke 1.2

- static KeyStroke getKeyStroke(char keyChar)
Создает экземпляр класса KeyStroke, инкапсулирующий нажатие клавиши, которое соответствует событию KEY_TYPED.
- static KeyStroke getKeyStroke(int keyCode, int modifiers)
- static KeyStroke getKeyStroke(int keyCode, int modifiers, boolean onRelease)

Создает объект класса `KeyStroke`, инкапсулирующий нажатие клавиши, которое соответствует событию `KEY_PRESSED` или `KEY_RELEASED`.

Параметры	<code>keyCode</code>	Виртуальный код клавиши
	<code>modifiers</code>	Одно из сочетаний: <code>InputEvent.SHIFT_MASK</code> , <code>InputEvent.CONTROL_MASK</code> , <code>InputEvent.ALT_MASK</code> или <code>InputEvent.META_MASK</code>
	<code>onRelease</code>	Значение <code>true</code> , если код соответствует отпущенной клавише

- `static KeyStroke getKeyStroke(String description)`

Создает объект класса `KeyStroke` по описанию, представленному в виде строки. Строка состоит из лексем, разделенных пробелами, в следующем формате.

1. Лексемы `shift control ctrl meta alt button1 button2 button3` преобразовываются в соответствующую битовую маску.
2. Лексема `typed` должна сопровождаться символом, например `"typed a"`.
3. Лексема `pressed` или `released` означает, что клавиша нажата или отпущена. (По умолчанию предполагается, что клавиша нажата.)
4. В противном случае лексема с префиксом `VK_` должна соответствовать константе `KeyEvent`, например `INSERT` соответствует `KeyEvent.VK_INSERT`.

Так, строка `"released ctrl Y"` соответствует выражению `getKeyStroke(KeyEvent.VK_Y, Event.CTRL_MASK, true)`.



javax.swing.JComponent 1.2

- `ActionMap getActionMap()` 1.3

Возвращает отображения клавиш в действия.

- `InputMap getInputMap(int flag)` 1.3

Получает отображение ввода, которое переводит ключи действий в объекты действий.

Параметр `flag` Одно из значений, представленных в табл. 8.4

Многоадресная передача событий

В предыдущем разделе мы связали несколько источников событий с одним обработчиком. Теперь попробуем сделать противоположное. Все источники событий в библиотеке AWT поддерживают модель *многоадресной передачи событий* (multicast model). Это означает, что одно и то же событие может передаваться нескольким объектам-обработчикам. Многоадресная модель передачи событий полезна, когда событие *потенциально* представляет интерес для нескольких объектов. Нужно лишь добавить несколько обработчиков для одного и того же источника событий, и все зарегистрированные объекты смогут реагировать на эти события.

Рассмотрим простое приложение, в котором используется многоадресная передача событий. Предположим, что, щелкая на кнопке `New`, в некоем фрейме, пользователь открыл несколько окон. Все окна закрываются кнопкой `Close all` (рис. 8.10).



Как сказано в документации JDK, “API не гарантирует определенного порядка, в котором события передаются обработчикам, зарегистрированным для данного источника”. Поэтому нельзя при создании программ реализовывать логику, зависящую от последовательности передачи событий.

Обработчик события, связанного с кнопкой **New**, — объект `newListener`, созданный конструктором класса `MulticastPanel`, — формирует новый фрейм в ответ на щелчок на кнопке.

Однако кнопка **Close all** класса `MulticastPanel` имеет несколько обработчиков. При каждом вызове метод `makeNewFrame()` добавляет к кнопке **Close all** еще одного обработчика. Когда пользователь щелкает на кнопке **Close all**, активизируются все обработчики, и каждый из них закрывает свой фрейм.

Метод `actionPerformed()` удаляет обработчик, связанный ранее с кнопкой **Close**, поскольку в нем больше нет необходимости (фрейм закрыт).

Полный текст программы приведен в листинге 8.6.



Рис. 8.10. Все фреймы реагируют на команду **Close all**

Листинг 8.6. Содержимое файла MulticastTest.java

```

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class MulticastTest
{
    public static void main(String[] args)
    {
        MulticastFrame frame = new MulticastFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм с кнопками, предназначенными для создания
 * и удаления вторичных фреймов.
 */
class MulticastFrame extends JFrame
{
    public MulticastFrame()
    {
        setTitle("MulticastTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        MulticastPanel panel = new MulticastPanel();
        add(panel);
    }

    public static final int DEFAULT_WIDTH = 300;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель с кнопками для создания и удаления фреймов.
 */
class MulticastPanel extends JPanel
{
    public MulticastPanel()
    {
        // Добавление кнопки New

        JButton newButton = new JButton("New");
        add(newButton);
        final JButton closeAllButton = new JButton("Close all");
        add(closeAllButton);

        ActionListener newListener = new
            ActionListener()
            {
                public void actionPerformed(ActionEvent event)
                {

```

```

        BlankFrame frame = new BlankFrame(closeAllButton);
        frame.setVisible(true);
    };
}

newButton.addActionListener(newListener);
}
}

/**
 * Пустой фрейм, который закрывается щелчком на кнопке.
 */
class BlankFrame extends JFrame
{
    /**
     * Создание пустого фрейма.
     * @param closeButton Кнопка для закрытия фрейма
     */
    public BlankFrame(final JButton closeButton)
    {
        counter++;
        setTitle("Frame " + counter);
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
        setLocation(SPACING * counter, SPACING * counter);

        closeListener = new
            ActionListener()
            {
                public void actionPerformed(ActionEvent event)
                {
                    closeButton.removeActionListener(closeListener);
                    dispose();
                }
            };
        closeButton.addActionListener(closeListener);
    }

    private ActionListener closeListener;
    public static final int DEFAULT_WIDTH = 200;
    public static final int DEFAULT_HEIGHT = 150;
    public static final int SPACING = 40;
    private static int counter = 0;
}

```

Реализация источников событий

В последнем разделе данной главы мы покажем, как реализуется класс, генерирующий события и оповещающий о них обработчики. Такую задачу приходится решать при использовании сложных компонентов Swing. Как бы то ни было, каждому программисту должен быть интересен механизм связывания обработчиков с компонентами.

В качестве источника событий будет выступать объект `PaintCountPanel`, подсчитывающий число вызовов метода `paintComponent`. При каждом увеличении счетчика `PaintCountPanel` оповещает всех обработчиков. В нашем примере мы свяжем с источником один обработчик, который будет изменять заголовок фрейма (рис. 8.11).

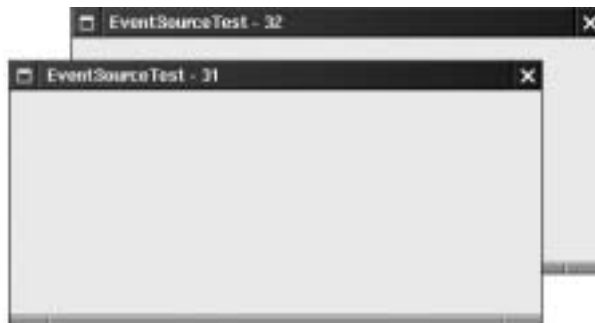


Рис. 8.11. Подсчет числа перерисовок панели

Определяя источник событий, надо указать следующие сведения.

- *Тип события.* Мы можем определить собственный класс события, но для простоты воспользуемся существующим классом `PropertyChangeEvent`.
- *Интерфейс обработчика.* Допустимо определить собственный интерфейс, но мы используем существующий интерфейс `PropertyChangeListener`. В нем объявлен единственный метод.

```
public void propertyChange(PropertyChangeEvent event)
```

- *Методы для связывания обработчиков и их удаления.* В классе `PaintCountPanel` мы определим следующие два метода:

```
public void addPropertyChangeListener
    (PropertyChangeListener listener)
public void removePropertyChangeListener
    (PropertyChangeListener listener)
```

Как убедиться в том, что информация о событиях передана обработчикам? За оповещение отвечает источник. Именно он при возникновении события должен создать соответствующий объект и передать его зарегистрированным обработчикам.

Задачу управления событиями приходится решать достаточно часто, поэтому в библиотеке Swing предусмотрен класс `EventListenerList`. Этот класс упрощает добавление и удаление обработчиков, а также генерацию событий. Он также берет на себя всю сложную работу, связанную с обработкой нескольких потоков, пытающихся одновременно добавлять, удалять или передавать события.

Поскольку некоторые источники событий допускают использование нескольких типов обработчиков, каждый обработчик в списке событий соответствует конкретному классу. Методы `add()` и `remove()` предназначены для реализации методов `addXxxListener`.

```
public void addPropertyChangeListener
    (PropertyChangeListener listener)
{
    listenerList.add(PropertyChangeListener.class, listener);
}
```



```
public void removePropertyChangeListener
(PropertyChangeListener listener)
{
    listenerList.remove(PropertyChangeListener.class, listener);
}
```



Может возникнуть вопрос, почему класс `EventListenerList` не проверяет, какой интерфейс реализуется объектом обработчика. Однако объект может реализовывать несколько интерфейсов. Например, может оказаться, что объект, на который ссылается параметр `listener`, одновременно реализует интерфейсы `PropertyChangeListener` и `ActionListener`, а программист захочет, чтобы при вызове метода `addPropertyChangeListener()` добавлялся только обработчик `PropertyChangeListener`. Класс `EventListenerList` должен обеспечить возможность такого выбора.

При вызове метода `paintComponent()` класс `PaintCountPanel` создает объект `PropertyChangeEvent`, указывая в нем источник события, имя свойства, его старое и новое значения. Затем происходит вызов вспомогательного метода `firePropertyChangeEvent()`.

```
public void paintComponent(Graphics g)
{
    int oldPaintCount = paintCount;
    paintCount++;
    firePropertyChangeEvent(new PropertyChangeEvent(this,
        "paintCount", oldPaintCount, paintCount));
    super.paintComponent(g);
}
```

Метод `firePropertyChangeEvent()` обнаруживает всех зарегистрированных обработчиков и для каждого из них вызывает метод `propertyChange()`.

```
public void firePropertyChangeEvent(PropertyChangeEvent event)
{
    EventListener[] listeners =
        listenerList.getListeners(PropertyChangeListener.class);
    for (EventListener l : listeners)
        ((PropertyChangeListener) l).propertyChange(event);
}
```

В листинге 8.7 приведен исходный код программы, основу которой составляет класс `PaintCountPanel`. Конструктор фрейма связывает с панелью обработчик, реагирующий на изменение свойства. Этот обработчик обновляет строку заголовка фрейма.

```
panel.addPropertyChangeListener(new
    PropertyChangeListener()
    {
        public void propertyChange(PropertyChangeEvent event)
        {
            setTitle("EventSourceTest - " + event.getNewValue());
        }
    });
```

Этим примером мы заканчиваем обсуждение обработки событий. В следующей главе подробнее описываются компоненты пользовательского интерфейса. Разумеется, что-

бы запрограммировать пользовательский интерфейс, необходимо знать механизм обработки событий, поскольку действия большинства программ выполняются именно в ответ на события, генерируемые компонентами пользовательского интерфейса.

Листинг 8.7. Содержимое файла `CustomEventTest.java`

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import javax.swing.*;
import java.beans.*;

public class EventSourceTest
{
    public static void main(String[] args)
    {
        EventSourceFrame frame = new EventSourceFrame();
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

/**
 * Фрейм, содержащий панель, которая допускает рисование.
 */
class EventSourceFrame extends JFrame
{
    public EventSourceFrame()
    {
        setTitle("EventSourceTest");
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);

        // Добавление панели к фрейму.

        final PaintCountPanel panel = new PaintCountPanel();
        add(panel);

        panel.addPropertyChangeListener(new
            PropertyChangeListener()
            {
                public void propertyChange(PropertyChangeEvent event)
                {
                    setTitle("EventSourceTest - " +
                        event.getNewValue());
                }
            });
    }

    public static final int DEFAULT_WIDTH = 400;
    public static final int DEFAULT_HEIGHT = 200;
}

/**
 * Панель, подсчитывающая число перерисовок.
 */
```

```

class PaintCountPanel extends JPanel
{
    public void paintComponent(Graphics g)
    {
        int oldPaintCount = paintCount;
        paintCount++;
        firePropertyChangeEvent(new PropertyChangeEvent(this,
            "paintCount", oldPaintCount, paintCount));
        super.paintComponent(g);
    }

    /**
     * Добавление обработчика событий.
     * @param listener Обработчик
     */
    public void addPropertyChangeListener
        (PropertyChangeListener listener)
    {
        listenerList.add(PropertyChangeListener.class, listener);
    }

    /**
     * Удаление обработчика событий.
     * @param listener Обработчик
     */
    public void removePropertyChangeListener
        (PropertyChangeListener listener)
    {
        listenerList.remove(PropertyChangeListener.class, listener);
    }

    public void firePropertyChangeEvent(PropertyChangeEvent event)
    {
        EventListener[] listeners =
            listenerList.getListeners
                (PropertyChangeListener.class);
        for (EventListener l : listeners)
            ((PropertyChangeListener) l).propertyChange(event);
    }

    public int getPaintCount()
    {
        return paintCount;
    }

    private int paintCount;
}

```



javax.swing.event.EventListenerList 1.2

- `void add(Class t, EventListener l)`

Добавляет обработчик событий и его класс в список. Класс хранится таким образом, чтобы методы, генерирующие события, могли выбирать требуемый класс. Обычно данный метод используется при построении метода `addXxxListener()`.

```
public void addXxxListener(XxxListener l)
{
    listenerList.add(XxxListener.class, l);
}
```

Параметры

t	Тип обработчика
l	Обработчик

- void remove(Class t, EventListener l)

Удаляет обработчик события и его класс из списка. Обычно данный метод используется для создания метода removeXxxListener().

```
public void removeXxxListener(XxxListener l)
{
    listenerList.remove(XxxListener.class, l);
}
```

Параметры

t	Тип обработчика
l	Обработчик

- EventListener[] getListeners(Class t) **1.3**

Возвращает массив, содержащий всех обработчиков заданного типа. Гарантируется, что метод не вернет значение null.

- Object[] getListenerList()

Возвращает массив, в котором элементами с четными индексами являются классы, а с нечетными — объекты-обработчики. Гарантируется, что метод не вернет значение null.



java.beans.PropertyChangeEvent 1.1

- PropertyChangeEvent(Object source, String name, Object oldValue, Object newValue)

Создает событие, соответствующее изменению свойства.

Параметры

source	Источник события — объект, сообщающий об изменении свойства
name	Имя свойства
oldValue	Значение свойства до изменения
newValue	Значение свойства после изменения



java.beans.PropertyChangeListener 1.1

- void propertyChange(PropertyChangeEvent event)

Вызывается при изменении значения свойства.